

Annotated Training Code PDF

A4 landscape. Left column preserves source code with line numbers; right column gives colored Chinese annotations for each non-continuation code row. The four chapters keep MLP, DANN, official CDAN, and custom C-DANN method boundaries separate.

1. Chapter 1 - MLP baseline and shared ML training pipeline

This chapter preserves the full ML baseline script. The MLP branch is defined through MODELS, GRIDS, DEFAULT_PARAMS, make_estimator(), fit_estimator(), select_params(), and evaluate_split().

2. Chapter 2 - DANN training

This chapter contains the DANN feature extractor G_f , label classifier G_y , domain classifier G_d , gradient reversal, split construction, training loop, and model selection.

3. Chapter 3 - Official CDAN training

This chapter contains official CDAN-style conditional adversarial training with $T(h,g)=g$ outer h and a conditional domain discriminator G_{cd} .

4. Chapter 4 - Custom C-DANN centroid alignment

This chapter contains the custom C-DANN variant that adds class-wise centroid alignment L_c to the DANN objective. It is intentionally separated from official CDAN.

Chapter 1 - MLP baseline and shared ML training pipeline

This chapter preserves the full ML baseline script. The MLP branch is defined through MODELS, GRIDS, DEFAULT_PARAMS, make_estimator(), fit_estimator(), select_params(), and evaluate_split().

Source file: D:\thesis\code\training\run_manifest_da_baseline_splits.py. Total lines: 935. The following pages preserve the source order and line numbers.

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
1 from __future__ import annotations
2
3 import os
4 import re
5 import warnings
6 from pathlib import Path
7
8 import numpy as np
9 import pandas as pd
10 from sklearn.base import clone
11 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
12 from sklearn.ensemble import RandomForestClassifier
13 from sklearn.exceptions import ConvergenceWarning
14 from sklearn.linear_model import LogisticRegression
15 from sklearn.metrics import balanced_accuracy_score, confusion_matrix, f1_score, precision_recall_fscore_support
16 from sklearn.neighbors import KNeighborsClassifier
17 from sklearn.neural_network import MLPClassifier
18 from sklearn.pipeline import Pipeline
19 from sklearn.preprocessing import StandardScaler
20 from sklearn.svm import SVC
21
22
23 OUT_ROOT = Path(r"D:\thesis")
24 TABLE_DIR = OUT_ROOT / "tables"
25 FIG_DIR = OUT_ROOT / "figures" / "ml_learning"
26 DATA_ROOT = OUT_ROOT / "enose_data"
27 FEATURE_PATH = TABLE_DIR / "manifest_baseline_as_air_features.csv"
28 RAW_MANIFEST_PATH = TABLE_DIR / "manifest_reconstructed_from_raw.csv"
29
30 SENSORS = [f"s{i}" for i in range(1, 7)]
31 THREE_CLASSES = ["air", "alcohol", "acetone"]
32 MODELS = ["LDA", "Logistic Regression", "kNN", "Random Forest", "RBF SVM", "MLP"]
33 FINAL_TRAINING = os.environ.get("ENOSE_FINAL_TRAINING", "train_only").strip().lower()
34 REBUILD_FEATURES_FROM_RAW = os.environ.get("ENOSE_REBUILD_FEATURES_FROM_RAW", "1").strip() == "1"
35
36 GRIDS = {
37     "LDA": [{"shrinkage": v} for v in [0.01, 0.1, 0.25, 0.5]],
38     "Logistic Regression": [{"C": v} for v in [0.1, 1.0, 10.0]],
39     "kNN": [{"n_neighbors": v} for v in [1, 3, 5, 7]],
40     "Random Forest": [
41         {"max_features": mf, "max_depth": depth, "min_samples_leaf": leaf}
42         for mf in ["sqrt", "log2"]
43         for depth in [2, 3, 4, None]
44         for leaf in [1, 2, 4]
45     ],
46     "RBF SVM": [{"C": c, "gamma": g} for c in [0.1, 1.0, 10.0] for g in ["scale", 0.001, 0.01]],
47     "MLP": [{"hidden_layer_sizes": (h,), "alpha": alpha, "learning_rate_init": lr} for h in [8, 16, 32] for alpha in [0.001, 0.01] for lr
48             ... in [0.001, 0.01]],
49 }
50
51 DEFAULT_PARAMS = {
52     "LDA": {"shrinkage": 0.1},
53     "Logistic Regression": {"C": 1.0},
54     "kNN": {"n_neighbors": 3},
55     "Random Forest": {"max_features": "sqrt", "max_depth": 4, "min_samples_leaf": 2},
56     "RBF SVM": {"C": 1.0, "gamma": "scale"},
57     "MLP": {"hidden_layer_sizes": (16,), "alpha": 0.001, "learning_rate_init": 0.001},
58 }
```

启用新版 Python 注解行为，避免类型注解在运行时过早求值。

空行，用来分隔逻辑块，保持代码结构可读。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

导入本项目需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

循环遍历 split、模型、超参数、epoch 或样本记录。

循环遍历 split、模型、超参数、epoch 或样本记录。

循环遍历 split、模型、超参数、epoch 或样本记录。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

上一行过长后的续行，保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
58
59 SPLITS = [
60     {
61         "split": "S1_train_I_test_all",
62         "description": "Train Period I only; no validation; test all Period II/III days.",
63         "train": lambda df: df["period"].eq("Period_I"),
64         "val": lambda df: pd.Series(False, index=df.index),
65         "test": lambda df: df["period"].isin(["Period_II", "Period_III"]),
66     },
67     {
68         "split": "S2_train_I_internal_val_test_all",
69         "description": "Train a stratified random Period I holdout split; three-class uses train 25 / validation 5; test all Period
        ... II/III days.",
70         "train": lambda df: period_i_internal_mask(df, "train"),
71         "val": lambda df: period_i_internal_mask(df, "validation"),
72         "test": lambda df: df["period"].isin(["Period_II", "Period_III"]),
73     },
74     {
75         "split": "S3_train_I_internal_20_10_test_all",
76         "description": "Train a stratified random Period I holdout split; three-class uses train 20 / validation 10; test all Period
        ... II/III days.",
77         "train": lambda df: period_i_internal_mask(df, "train", "20_10"),
78         "val": lambda df: period_i_internal_mask(df, "validation", "20_10"),
79         "test": lambda df: df["period"].isin(["Period_II", "Period_III"]),
80     },
81     {
82         "split": "S4_train_I_val_D1_2",
83         "description": "Train Period I; validate Period II Day1-Day2; test Period II Day3-Day17 + Period III Day18-Day19.",
84         "train": lambda df: df["period"].eq("Period_I"),
85         "val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(1, 2),
86         "test": lambda df: (
87             (df["period"].eq("Period_II") & df["day_num"].between(3, 17))
88             | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
89         ),
90     },
91     {
92         "split": "S5_train_I_D1_val_D2_3",
93         "description": "Train Period I + Period II Day1; validate Period II Day2-Day3; test Period II Day4-Day17 + Period III
        ... Day18-Day19.",
94         "train": lambda df: df["period"].eq("Period_I") | (df["period"].eq("Period_II") & df["day_num"].eq(1)),
95         "val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(2, 3),
96         "test": lambda df: (
97             (df["period"].eq("Period_II") & df["day_num"].between(4, 17))
98             | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
99         ),
100     },
101     {
102         "split": "S6_train_I_D1_2_val_D3_5",
103         "description": "Train Period I + Period II Day1-Day2; validate Period II Day3-Day5; test Period II Day6-Day17 + Period III
        ... Day18-Day19.",
104         "train": lambda df: df["period"].eq("Period_I") | (df["period"].eq("Period_II") & df["day_num"].between(1, 2)),
105         "val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(3, 5),
106         "test": lambda df: (
107             (df["period"].eq("Period_II") & df["day_num"].between(6, 17))
108             | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
109         ),
110     },
111 ]
```

空行, 用来分隔逻辑块, 保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

保存中间变量, 供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

上一行过长后的续行, 保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

上一行过长后的续行, 保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

上一行过长后的续行, 保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

上一行过长后的续行, 保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```

112 "split": "S7_train_I_D1_3_val_D4_5",
113 "description": "Train Period I + Period II Day1-Day3; validate Period II Day4-Day5; test Period II Day6-Day17 + Period III
... Day18-Day19.",
114 "train": lambda df: df["period"].eq("Period_I") | (df["period"].eq("Period_II") & df["day_num"].between(1, 3)),
115 "val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(4, 5),
116 "test": lambda df: (
117     (df["period"].eq("Period_II") & df["day_num"].between(6, 17))
118     | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
119 ),
120 },
121 ]
122
123 TASKS = [
124     ("three_class_with_air", THREE_CLASSES),
125 ]
126
127
128 def natural_key(value: str) -> tuple:
129     parts = re.split(r"(\d+)", str(value))
130     return tuple(int(part) if part.isdigit() else part for part in parts)
131
132
133 def period_i_internal_mask(df: pd.DataFrame, partition: str, scheme: str = "25_5") -> pd.Series:
134     mask = pd.Series(False, index=df.index)
135     period_i = df[df["period"].eq("Period_I")]
136     labels = set(period_i["label"].unique())
137     if scheme == "20_10":
138         if {"air", "alcohol", "acetone"}.issubset(labels):
139             validation_counts = {"air": 2, "alcohol": 4, "acetone": 4}
140         else:
141             validation_counts = {"alcohol": 4, "acetone": 4}
142     else:
143         if {"air", "alcohol", "acetone"}.issubset(labels):
144             validation_counts = {"air": 1, "alcohol": 2, "acetone": 2}
145         else:
146             validation_counts = {"alcohol": 2, "acetone": 2}
147     for label, sub in period_i.groupby("label"):
148         ordered = sorted(sub.index, key=lambda idx: natural_key(df.loc[idx, "target_file"]))
149         label_rng = np.random.default_rng(42 + sum(ord(ch) for ch in str(label)))
150         ordered = list(label_rng.permutation(ordered))
151         val_n = min(validation_counts.get(str(label), 0), max(0, len(ordered) - 1))
152         val_idx = set(ordered[:val_n])
153         selected = [idx for idx in ordered if idx not in val_idx] if partition == "train" else list(val_idx)
154         mask.loc[selected] = True
155     return mask
156
157
158 def feature_columns(df: pd.DataFrame) -> list[str]:
159     return [c for c in df.columns if re.match(r"s\d+_", c)]
160
161
162 def raw_path_for_metadata(row: pd.Series) -> Path:
163     label = str(row["label"])
164     target_file = str(row["target_file"]).replace("\\", "/")
165     stem = Path(target_file).stem
166     if label in {"alcohol", "acetone"}:
167         old_match = re.fullmatch(fr"{label}_o(\d+)", stem)
168         new_match = re.fullmatch(fr"{label} (\d+)", stem)

```

定义时间/域划分，明确训练、验证、测试样本来源。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

上一行过长的续行，保持同一条代码的剩余内容。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

空行，用来分隔逻辑块，保持代码结构可读。

窄段函数来分隔逻辑块，保持代码结构可读。

natural_key，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

窄段函数来分隔逻辑块，保持代码结构的训练的、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

窄段函数来分隔逻辑块，保持代码结构可读。

feature_columns，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

窄段函数来分隔逻辑块，保持代码结构的训练的、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
169     if old_match:
170         return DATA_ROOT / label / "enose_data_old" / f"{label}_{old_match.group(1)}.csv"
171     if new_match:
172         return DATA_ROOT / label / "enose_data" / f"{label}_{new_match.group(1)}.csv"
173     if label == "air":
174         old_match = re.fullmatch(r"air_ob(\d+)", stem)
175         new_match = re.fullmatch(r"air_(\d+)", stem)
176         ref_match = re.fullmatch(r"air_ref(\d+)", stem)
177         if old_match:
178             return DATA_ROOT / "air" / "enose_data_old" / "baseline" / f"baseline_{old_match.group(1)}.csv"
179         if new_match:
180             return DATA_ROOT / "air" / "enose_data" / "air" / f"air_{new_match.group(1)}.csv"
181         if ref_match:
182             return DATA_ROOT / "air" / "enose_data" / "reference" / f"reference_{ref_match.group(1)}.csv"
183     raise ValueError(f"Cannot map metadata target_file to a raw CSV path: label={label}, target_file={target_file}")
184
185
186 def build_reconstructed_manifest_from_metadata(metadata: pd.DataFrame) -> pd.DataFrame:
187     required = ["label", "target_file", "period", "day", "day_num"]
188     missing = [col for col in required if col not in metadata.columns]
189     if missing:
190         raise ValueError(f"Metadata table is missing required columns for manifest reconstruction: {missing}")
191     rows = []
192     for row in metadata[required].drop_duplicates().to_dict("records"):
193         series = pd.Series(row)
194         raw_path = raw_path_for_metadata(series)
195         if not raw_path.exists():
196             raise FileNotFoundError(f"Raw CSV required by reconstructed manifest does not exist: {raw_path}")
197         rows.append(
198             {
199                 **row,
200                 "raw_path": str(raw_path),
201             }
202         )
203     manifest = pd.DataFrame(rows).sort_values(["label", "period", "day_num", "target_file"], key=lambda s: s.map(str)
204         ... ).reset_index(drop=True)
205     expected_counts = {"acetone": 92, "air": 73, "alcohol": 92}
206     counts = manifest["label"].value_counts().to_dict()
207     if counts != expected_counts:
208         raise ValueError(f"Reconstructed manifest label counts do not match expected raw data counts: {counts} != {expected_counts}")
209     return manifest
210
211 def load_or_create_reconstructed_manifest() -> pd.DataFrame:
212     if RAW_MANIFEST_PATH.exists():
213         manifest = pd.read_csv(RAW_MANIFEST_PATH)
214     else:
215         if not FEATURE_PATH.exists():
216             raise FileNotFoundError(
217                 f"{RAW_MANIFEST_PATH} does not exist and {FEATURE_PATH} is unavailable for one-time metadata reconstruction."
218             )
219         metadata = pd.read_csv(FEATURE_PATH)
220         manifest = build_reconstructed_manifest_from_metadata(metadata)
221         manifest.to_csv(RAW_MANIFEST_PATH, index=False)
222     required = ["label", "target_file", "period", "day", "day_num", "raw_path"]
223     missing = [col for col in required if col not in manifest.columns]
224     if missing:
225         raise ValueError(f"Reconstructed manifest is missing required columns: {missing}")
```

条件分支，按数据可用性、划分策略或方法变体选择严格路径。
指定输入特征表、原始数据目录或输出表格/图像位置。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
指定输入特征表、原始数据目录或输出表格/图像位置。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
指定输入特征表、原始数据目录或输出表格/图像位置。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
指定输入特征表、原始数据目录或输出表格/图像位置。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
指定输入特征表、原始数据目录或输出表格/图像位置。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
空行，用来分隔逻辑块，保持代码结构可读。
空行/函数未分隔逻辑块，保持代码结构可读。
指定输入特征表、原始数据目录或输出表格/图像位置。
评估或数据处理步骤独立出来。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
保存中间变量，供后续训练、评估或输出步骤使用。
上一行过长后的续行，保持同一条代码的剩余内容。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
返回本函数计算出的模型、指标、表格行或中间结果。
空行，用来分隔逻辑块，保持代码结构可读。
空行/函数未分隔逻辑块，保持代码结构可读。
指定输入特征表、原始数据目录或输出表格/图像位置。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
指定输入特征表、原始数据目录或输出表格/图像位置。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
指定输入特征表、原始数据目录或输出表格/图像位置。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
指定输入特征表、原始数据目录或输出表格/图像位置。
保存中间变量，供后续训练、评估或输出步骤使用。
写出结果表、图或日志，供论文图表和复查使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
226 missing_files = [path for path in manifest["raw_path"].astype(str) if not Path(path).exists()]
227 if missing_files:
228     raise FileNotFoundError(f"Reconstructed manifest references missing raw files: {missing_files[:10]}")
229 return manifest
230
231
232 def elapsed_seconds(raw: pd.DataFrame, path: Path) -> np.ndarray:
233     if "arduino_time" in raw.columns:
234         t = pd.to_numeric(raw["arduino_time"], errors="raise").to_numpy(float)
235         return (t - t[0]) / 1000.0
236     if "time_s" in raw.columns:
237         t = pd.to_numeric(raw["time_s"], errors="raise").to_numpy(float)
238         return t - t[0]
239     raise ValueError(f"{path} has neither arduino_time nor time_s.")
240
241
242 def load_raw_measurements(path: Path) -> pd.DataFrame:
243     raw = pd.read_csv(path)
244     time_cols = [col for col in ["arduino_time", "time_s"] if col in raw.columns]
245     if not time_cols:
246         raise ValueError(f"{path} has neither arduino_time nor time_s.")
247     required = time_cols + SENSORS
248     missing = [col for col in required if col not in raw.columns]
249     if missing:
250         raise ValueError(f"{path} is missing required columns: {missing}")
251     numeric_required = raw[required].apply(pd.to_numeric, errors="coerce")
252     empty_rows = numeric_required.isna().all(axis=1)
253     partial_invalid_rows = numeric_required.isna().any(axis=1) & ~empty_rows
254     if partial_invalid_rows.any():
255         bad_rows = raw.index[partial_invalid_rows].tolist()[:10]
256         raise ValueError(f"{path} has non-numeric or partially missing measurement rows: {bad_rows}")
257     if empty_rows.any():
258         raw = raw.loc[~empty_rows].copy()
259     if raw.empty:
260         raise ValueError(f"{path} has no valid measurement rows.")
261     return raw
262
263
264 def phase_masks(label: str, duration: float, t: np.ndarray, path: Path) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
265     if len(t) == 0:
266         raise ValueError(f"{path} has no time samples.")
267     total = float(np.max(t))
268     if label == "air":
269         baseline_end = min(60.0, max(20.0, total * 0.35))
270         exposure_start = baseline_end
271         exposure_end = total
272         recovery_start = max(exposure_start, total * 0.70)
273     else:
274         baseline_end = min(60.0, max(5.0, total * 0.45))
275         exposure_start = 60.0
276         if duration >= 175:
277             exposure_end = 120.0
278         elif duration >= 128:
279             exposure_end = 70.0
280         elif duration >= 123:
281             exposure_end = 65.0
282     else:
283         exposure_end = min(total, exposure_start + max(1.0, duration - 120.0))
```

保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
返回本函数计算出的模型、指标、表格行或中间结果。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
elapsed_seconds，把可复用的训练、评估或数据处理步骤独立出来。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
返回本函数计算出的模型、指标、表格行或中间结果。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
返回本函数计算出的模型、指标、表格行或中间结果。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
返回本函数计算出的模型、指标、表格行或中间结果。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
phase_masks，把可复用的训练、评估或数据处理步骤独立出来。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。

D:\thesis\code\training\run_manifest_da_baseline_splits.py

[illegible]

D:\thesis\code\training\run_manifest_da_baseline_splits.py

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构清晰，可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

MLP 基线训练流程中的普通语句，维持数据流或控制流。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构清晰，可复用的训练、评估或数据处理步骤独立出来。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构可读。

validate split parts，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构可读。

require classes，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构清晰，可复用的训练、评估或数据处理步骤独立出来。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

循环遍历 split、模型、超参数、epoch 或样本记录。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 encode，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
400     return np.array([mapping[str(label)] for label in labels], dtype=int)
401
402
403 def metric_dict(y_true: np.ndarray, y_pred: np.ndarray, n_classes: int) -> dict:
404     if len(y_true) == 0:
405         raise ValueError("Cannot compute metrics on an empty evaluation set.")
406     labels = list(range(n_classes))
407     cm = confusion_matrix(y_true, y_pred, labels=labels)
408     return {
409         "accuracy": float((y_true == y_pred).mean()),
410         "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)),
411         "macro_f1": float(f1_score(y_true, y_pred, labels=labels, average="macro", zero_division=0)),
412         "confusion": cm,
413     }
414
415
416 def partition_metric_records(
417     split_name: str,
418     task_name: str,
419     model_name: str,
420     partition: str,
421     y_true: np.ndarray,
422     y_pred: np.ndarray,
423     n_classes: int,
424 ) -> dict:
425     metrics = metric_dict(y_true, y_pred, n_classes)
426     return {
427         "split": split_name,
428         "task": task_name,
429         "model": model_name,
430         "partition": partition,
431         "n": len(y_true),
432         "accuracy": metrics["accuracy"],
433         "balanced_accuracy": metrics["balanced_accuracy"],
434         "macro_f1": metrics["macro_f1"],
435     }
436
437
438 def confusion_records(
439     split_name: str,
440     task_name: str,
441     model_name: str,
442     partition: str,
443     classes: list[str],
444     y_true: np.ndarray,
445     y_pred: np.ndarray,
446 ) -> list[dict]:
447     cm = confusion_matrix(y_true, y_pred, labels=list(range(len(classes))))
448     rows = []
449     for i, true_label in enumerate(classes):
450         for j, pred_label in enumerate(classes):
451             rows.append(
452                 {
453                     "split": split_name,
454                     "task": task_name,
455                     "model": model_name,
456                     "partition": partition,
457                     "true_label": true_label,
```

返回本函数计算出的模型、指标、表格行或中间结果。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
metric_dict，把可复用的训练、评估或数据处理步骤独立出来。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
返回本函数计算出的模型、指标、表格行或中间结果。
保存中间变量，供后续训练、评估或输出步骤使用。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
返回本函数计算出的模型、指标、表格行或中间结果。
定义时间/域划分，明确训练、验证、测试样本来源。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
confusion_records，把可复用的训练、评估或数据处理步骤独立出来。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
循环遍历 split、模型、超参数、epoch 或样本记录。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
定义时间/域划分，明确训练、验证、测试样本来源。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。

D:\thesis\code\training\run_manifest_da_baseline_splits.py

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行/函数来分隔逻辑块, 保持代码结构可读。

class metric records, 把可复用的训练、评估或数据处理步骤独立出来。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

定义时间/域划分, 明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

MLP/基线训练流程中的普通语句, 维持数据流或控制流。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行/函数来分隔逻辑块, 保持代码结构可读。

空行/函数来分隔逻辑块, 保持代码结构可读。

make estimator, 把可复用的训练、评估或数据处理步骤独立出来。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支的备选路径, 仍保持明确规则而不是隐式猜测。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支的备选路径, 仍保持明确规则而不是隐式猜测。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
516     estimator = KNeighborsClassifier(n_neighbors=params["n_neighbors"])
517 elif model == "Random Forest":
518     estimator = RandomForestClassifier(
519         n_estimators=300,
520         max_features=params["max_features"],
521         class_weight="balanced",
522         random_state=41,
523         n_jobs=1,
524     )
525 elif model == "RBF SVM":
526     estimator = SVC(C=params["C"], gamma=params["gamma"], kernel="rbf", class_weight="balanced")
527 elif model == "MLP":
528     estimator = MLPClassifier(
529         hidden_layer_sizes=params["hidden_layer_sizes"],
530         alpha=params["alpha"],
531         learning_rate_init=params["learning_rate_init"],
532         activation="relu",
533         solver="adam",
534         max_iter=2000,
535         random_state=73,
536     )
537 else:
538     raise ValueError(f"Unknown model: {model}")
539 return Pipeline([("scale", StandardScaler()), ("model", estimator)])
540
541
542 def fit_estimator(estimator: Pipeline, x_train: np.ndarray, y_train: np.ndarray) -> Pipeline:
543     model = clone(estimator)
544     with warnings.catch_warnings():
545         warnings.filterwarnings("error", category=ConvergenceWarning)
546         model.fit(x_train, y_train)
547     return model
548
549
550 def select_params(
551     model_name: str,
552     x_train: np.ndarray,
553     y_train: np.ndarray,
554     x_val: np.ndarray,
555     y_val: np.ndarray,
556     n_classes: int,
557 ) -> tuple[dict, tuple[float, float, float]]:
558     best_params, best_score = None, None
559     for params in GRIDS[model_name]:
560         estimator = fit_estimator(make_estimator(model_name, params), x_train, y_train)
561         pred_val = estimator.predict(x_val)
562         val_metrics = metric_dict(y_val, pred_val, n_classes)
563         score = (val_metrics["macro_f1"], val_metrics["balanced_accuracy"], val_metrics["accuracy"])
564         if best_score is None or score > best_score:
565             best_score = score
566             best_params = params
567     if best_params is None or best_score is None:
568         raise ValueError(f"No hyperparameter candidate was evaluated for {model_name}.")
569     return best_params, best_score
570
571
572 def class_counts(df: pd.DataFrame, classes: list[str]) -> str:
573     counts = df["label"].value_counts().reindex(classes, fill_value=0)
```

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

构造 sklearn MLP 分类器，指定隐藏层、正则化、学习率和优化器。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

做标准化：脚本中会检查无效或近常数特征，避免静默错误。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

fit_estimator，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

保存中间变量，供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

select_params，把可复用的训练、评估或数据处理步骤独立出来。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

按验证指标选择超参数；无验证集时使用预声明固定参数。

定义传统 ML/MLP 的模型集合、搜索网格或无验证时默认参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

按验证指标选择超参数；无验证集时使用预声明固定参数。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

按验证指标选择超参数；无验证集时使用预声明固定参数。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

class_counts，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

D:\thesis\code\training\run_manifest_da_baseline_splits.py

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

定义时间/域划分来分隔逻辑块，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构可读。

assignment records”，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

定义时间/域划分，明确训练、验证、测试样本来源。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

MILP/基线训练流程中的普通语句，维持数据流或控制流。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数来分隔逻辑块，保持代码结构可读。

evaluate splits”，把可复用的训练、评估或数据处理步骤独立出来。

上一行过长的续行，保持同一条数据的剩余内容。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

定义时间/域划分，明确训练、验证、测试样本来源。

定义时间/域划分，明确训练、验证、测试样本来源。

定义时间/域划分，明确训练、验证、测试样本来源。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分，明确训练、验证、测试样本来源。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分，明确训练、验证、测试样本来源。

定义时间/域划分，明确训练、验证、测试样本来源。

定义时间/域划分，明确训练、验证、测试样本来源。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分，明确训练、验证、测试样本来源。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败用于：输入、划分或特征不满足实验假设时直接停止。

保存中间变量，供后续训练、评估或输出步骤使用。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
631 x_val_raw = val_df[cols].to_numpy(float)
632 x_test_raw = test_df[cols].to_numpy(float)
633 validate_feature_matrix(
634     x_train_raw,
635     {
636         "validation": x_val_raw,
637         "test": x_test_raw,
638     },
639 )
640 y_train = encode(train_df["label"].to_numpy(str), classes)
641 y_val = encode(val_df["label"].to_numpy(str), classes) if len(val_df) else np.array([], dtype=int)
642 y_test = encode(test_df["label"].to_numpy(str), classes)
643
644 split_rows = []
645 assignment_rows = []
646 for name, part_df in [("train", train_df), ("validation", val_df), ("test", test_df)]:
647     split_rows.append(
648         {
649             "split": split["split"],
650             "task": task_name,
651             "partition": name,
652             "n": len(part_df),
653             "class_counts": class_counts(part_df, classes) if len(part_df) else "",
654             "period_days": describe_period_days(part_df) if len(part_df) else "",
655         }
656     )
657     assignment_rows.extend(assignment_records(split["split"], task_name, name, part_df))
658
659 rows, cm_rows, partition_metric_rows, class_metric_rows = [], [], [], []
660 for model in MODELS:
661     if len(val_df) == 0:
662         best_params = DEFAULT_PARAMS[model]
663         selection_method = "fixed_default_no_validation"
664     else:
665         best_params, _best_score = select_params(model, x_train_raw, y_train, x_val_raw, y_val, len(classes))
666         selection_method = "validation_macro_f1_balanced_accuracy_accuracy"
667
668     selection_estimator = fit_estimator(make_estimator(model, best_params), x_train_raw, y_train)
669     pred_val = selection_estimator.predict(x_val_raw) if len(val_df) else np.array([], dtype=int)
670     val_metrics = metric_dict(y_val, pred_val, len(classes)) if len(val_df) else no_validation_metrics(len(classes))
671
672     if FINAL_TRAINING == "train_val_refit" and len(val_df):
673         final_df = pd.concat([train_df, val_df], ignore_index=True)
674         x_final_raw = final_df[cols].to_numpy(float)
675         y_final = encode(final_df["label"].to_numpy(str), classes)
676         require_classes(final_df, classes, split["split"], task_name, "final_train")
677         validate_feature_matrix(x_final_raw, {"test": x_test_raw})
678         final_estimator = fit_estimator(make_estimator(model, best_params), x_final_raw, y_final)
679         pred_train = final_estimator.predict(x_final_raw)
680         pred_test = final_estimator.predict(x_test_raw)
681         train_eval_n = len(final_df)
682     elif FINAL_TRAINING == "train_only":
683         pred_train = selection_estimator.predict(x_train_raw)
684         pred_test = selection_estimator.predict(x_test_raw)
685         y_final = y_train
686         train_eval_n = len(train_df)
687     else:
688         raise ValueError(f"Unknown ENOSE_FINAL_TRAINING value: {FINAL_TRAINING}")
```

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

定义时间/域划分，明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

定义时间/域划分，明确训练、验证、测试样本来源。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

定义传统 ML/MLP 的模型集合、搜索网格或无验证时默认参数。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义传统 ML/MLP 的模型集合、搜索网格或无验证时默认参数。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

按验证指标选择超参数；无验证集时使用预先声明固定参数。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

空行，用来分隔逻辑块，保持代码结构可读。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```

689 train_metrics = metric_dict(y_final, pred_train, len(classes))
690 test_metrics = metric_dict(y_test, pred_test, len(classes))
691
692
693 evaluated_partitions = [
694     ("train", y_final, pred_train),
695     ("test", y_test, pred_test),
696 ]
697 if len(val_df):
698     evaluated_partitions.insert(1, ("validation", y_val, pred_val))
699 for partition, y_true_part, y_pred_part in evaluated_partitions:
700     partition_metric_rows.append(
701         partition_metric_records(
702             split["split"],
703             task_name,
704             model,
705             partition,
706             y_true_part,
707             y_pred_part,
708             len(classes),
709         )
710     )
711     cm_rows.extend(
712         confusion_records(
713             split["split"],
714             task_name,
715             model,
716             partition,
717             classes,
718             y_true_part,
719             y_pred_part,
720         )
721     )
722     class_metric_rows.extend(
723         class_metric_records(
724             split["split"],
725             task_name,
726             model,
727             partition,
728             classes,
729             y_true_part,
730             y_pred_part,
731         )
732     )
733
734 rows.append(
735     {
736         "split": split["split"],
737         "description": split["description"],
738         "task": task_name,
739         "model": model,
740         "best_params": ";".join(f"{k}={v}" for k, v in best_params.items()),
741         "selection_method": selection_method,
742         "final_training": FINAL_TRAINING,
743         "train_n": train_eval_n,
744         "selection_train_n": len(train_df),
745         "validation_n": len(val_df),
746         "test_n": len(test_df),

```

[illegible]

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```

747         "train_accuracy": train_metrics["accuracy"],
748         "validation_accuracy": val_metrics["accuracy"],
749         "test_accuracy": test_metrics["accuracy"],
750         "train_balanced_accuracy": train_metrics["balanced_accuracy"],
751         "validation_balanced_accuracy": val_metrics["balanced_accuracy"],
752         "test_balanced_accuracy": test_metrics["balanced_accuracy"],
753         "train_macro_f1": train_metrics["macro_f1"],
754         "validation_macro_f1": val_metrics["macro_f1"],
755         "test_macro_f1": test_metrics["macro_f1"],
756     }
757 )
758 return rows, cm_rows, split_rows, partition_metric_rows, class_metric_rows, assignment_rows
759
760
761 def fmt(v: float) -> str:
762     return "NA" if pd.isna(v) else f"{v:.3f}"
763
764
765 def short_split(name: str) -> str:
766     return name.split("_", 1)[0]
767
768
769 def choose_best_by_test(results: pd.DataFrame) -> pd.DataFrame:
770     selected = []
771     for split_name, sub in results.groupby("split", sort=False):
772         ordered = sub.sort_values(
773             ["test_accuracy", "test_balanced_accuracy", "test_macro_f1", "model"],
774             ascending=[False, False, False, True],
775         )
776         row = ordered.iloc[0].copy()
777         row["summary_selection_note"] = "report-only best model by test metrics; not used for hyperparameter tuning"
778         selected.append(row)
779     if not selected:
780         raise ValueError("Cannot create selected summary because the ML results table is empty.")
781     return pd.DataFrame(selected).reset_index(drop=True)
782
783
784 def write_summary_svg(results: pd.DataFrame, path: Path) -> None:
785     cols = [
786         ("Split", 54),
787         ("Model", 122),
788         ("Best hyperparameters", 210),
789         ("Tr Acc", 56),
790         ("Tr BA", 56),
791         ("Tr F1", 56),
792         ("Val Acc", 56),
793         ("Val BA", 56),
794         ("Val F1", 56),
795         ("Test Acc", 62),
796         ("Test BA", 62),
797         ("Test F1", 62),
798         ("Selection", 205),
799     ]
800     row_h = 31
801     gap_h = 14
802     split_names = [s["split"] for s in SPLITS]
803     rows_h = row_h * (len(results) + 1) + gap_h * (len(split_names) - 1)
804     width = 32 + sum(w for _, w in cols) + 32

```

[illegible]

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
805 height = 118 + rows_h + 176
806 lines = [
807     f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width} {height}">',
808     '<rect width="100%" height="100%" fill="white"/>',
809     '<text x="24" y="32" font-family="Arial" font-size="22" font-weight="700">Machine-learning Baseline Multi-split Results for
      ... E-nose Drift</text>',
810     '<text x="24" y="56" font-family="Arial" font-size="13" fill="#555">Three-class only. Each row is trained on the declared split;
      ... validation selects hyperparameters when available. Test metrics are not used for tuning.</text>',
811     '<text x="24" y="74" font-family="Arial" font-size="12" fill="#555">Overfitting evidence is reported as raw train, validation,
      ... and test metrics. Class-level precision/recall/F1 and all partition confusion matrices are written to CSV.</text>',
812 ]
813
814 x0, y = 24, 92
815 x = x0
816 for label, w in cols:
817     lines.append(f'<rect x="{x}" y="{y}" width="{w}" height="{row_h}" fill="#f1f4f8" stroke="#d8dee8"/>')
818     lines.append(f'<text x="{x + 5}" y="{y + 20}" font-family="Arial" font-size="10" font-weight="700">{label}</text>')
819     x += w
820 y += row_h
821
822 for split_idx, split_name in enumerate(split_names):
823     sub = results[results["split"].eq(split_name)].copy()
824     if sub.empty:
825         raise ValueError(f"No rows available for split {split_name}.")
826     sub = sub.sort_values("model", key=lambda s: s.map({name: i for i, name in enumerate(MODELS)}))
827     for row_idx, (_, row) in enumerate(sub.iterrows()):
828         vals = [
829             short_split(row["split"]) if row_idx == 0 else "",
830             row["model"],
831             row["best_params"],
832             fmt(row["train_accuracy"]),
833             fmt(row["train_balanced_accuracy"]),
834             fmt(row["train_macro_f1"]),
835             fmt(row["validation_accuracy"]),
836             fmt(row["validation_balanced_accuracy"]),
837             fmt(row["validation_macro_f1"]),
838             fmt(row["test_accuracy"]),
839             fmt(row["test_balanced_accuracy"]),
840             fmt(row["test_macro_f1"]),
841             row["selection_method"],
842         ]
843         fill = "#ffffff" if row_idx % 2 == 0 else "#fbfcfe"
844         x = x0
845         for (_, label, w), val in zip(cols, vals):
846             lines.append(f'<rect x="{x}" y="{y}" width="{w}" height="{row_h}" fill="{fill}" stroke="#d8dee8"/>')
847             lines.append(f'<text x="{x + 5}" y="{y + 20}" font-family="Arial" font-size="9">{str(val)[:42]}</text>')
848             x += w
849         y += row_h
850     if split_idx != len(split_names) - 1:
851         lines.append(f'<rect x="{x0}" y="{y}" width="{sum(w for _, w in cols)}" height="{gap_h}" fill="#ffffff" stroke="none"/>')
852         y += gap_h
853
854 y += 22
855 legend = [
856     "S1: Train Period I only; no validation; test all Period II/III.",
857     "S2: Train stratified Period I subset; validate remaining Period I; test all Period II/III.",
858     "S3: Train stratified Period I subset; three-class train20/val10; test all Period II/III.",
859     "S4: Train Period I; validate Period II Day1-Day2; test Day3-Day19.",
```

保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
上一行过长后的续行，保持同一条代码的剩余内容。
保存中间变量，供后续训练、评估或输出步骤使用。
上一行过长后的续行，保持同一条代码的剩余内容。
保存中间变量，供后续训练、评估或输出步骤使用。
上一行过长后的续行，保持同一条代码的剩余内容。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
空行，用来分隔逻辑块，保持代码结构可读。
循环遍历 split、模型、超参数、epoch 或样本记录。
定义时间/域划分，明确训练、验证、测试样本来源。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
定义传统 ML/MLP 的模型集合、搜索网络或无验证时默认参数。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
定义时间/域划分，明确训练、验证、测试样本来源。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。
MLP/基线训练流程中的普通语句，维持数据流或控制流。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

```
860     "S5: Train Period I + Day1; validate Day2-Day3; test Day4-Day19.",
861     "S6: Train Period I + Day1-Day2; validate Day3-Day5; test Day6-Day19.",
862     "S7: Train Period I + Day1-Day3; validate Day4-Day5; test Day6-Day19.",
863     "Interpretation: inspect high train metrics together with lower validation/test metrics, unstable validation, and class-level
      ... failures; no numeric threshold is imposed here.",
864 ]
865 for line in legend:
866     lines.append(f'<text x="24" y="{y}" font-family="Arial" font-size="12" fill="#555">{line}</text>')
867     y += 17
868 lines.append("</svg>")
869 path.write_text("\n".join(lines), encoding="utf-8")
870
871 def main() -> None:
872     TABLE_DIR.mkdir(parents=True, exist_ok=True)
873     FIG_DIR.mkdir(parents=True, exist_ok=True)
874     if REBUILD_FEATURES_FROM_RAW:
875         features = build_feature_table_from_raw()
876         features.to_csv(FEATURE_PATH, index=False)
877         print(f"Rebuilt feature table from raw data: {FEATURE_PATH}")
878         print(f"Reconstructed raw manifest: {RAW_MANIFEST_PATH}")
879     else:
880         if not FEATURE_PATH.exists():
881             raise FileNotFoundError(f"Required feature table not found: {FEATURE_PATH}")
882         features = pd.read_csv(FEATURE_PATH)
883         print(f"Feature table: {FEATURE_PATH}")
884
885     all_rows, all_cm, all_splits, all_partition_metrics, all_class_metrics, all_assignments = [], [], [], [], [], []
886     for split in SPLITS:
887         for task_name, classes in TASKS:
888             rows, cm_rows, split_rows, partition_metric_rows, class_metric_rows, assignment_rows = evaluate_split(features, split,
889                 ... task_name, classes)
890             all_rows.extend(rows)
891             all_cm.extend(cm_rows)
892             all_splits.extend(split_rows)
893             all_partition_metrics.extend(partition_metric_rows)
894             all_class_metrics.extend(class_metric_rows)
895             all_assignments.extend(assignment_rows)
896
897     results = pd.DataFrame(all_rows)
898     confusions = pd.DataFrame(all_cm)
899     splits = pd.DataFrame(all_splits)
900     partition_metrics = pd.DataFrame(all_partition_metrics)
901     class_metrics = pd.DataFrame(all_class_metrics)
902     assignments = pd.DataFrame(all_assignments)
903     selected = choose_best_by_test(results)
904     results.to_csv(TABLE_DIR / "manifest_da_baseline_split_results.csv", index=False)
905     confusions.to_csv(TABLE_DIR / "manifest_da_baseline_split_confusions.csv", index=False)
906     splits.to_csv(TABLE_DIR / "manifest_da_baseline_split_splits.csv", index=False)
907     assignments.to_csv(TABLE_DIR / "manifest_da_baseline_split_assignments.csv", index=False)
908     partition_metrics.to_csv(TABLE_DIR / "manifest_da_baseline_partition_metrics.csv", index=False)
909     class_metrics.to_csv(TABLE_DIR / "manifest_da_baseline_class_metrics.csv", index=False)
910     selected.to_csv(TABLE_DIR / "manifest_da_baseline_selected_summary.csv", index=False)
911     write_summary_svg(results, FIG_DIR / "manifest_da_baseline_split_summary.svg")
912
913     cols = [
914         "split",
915         "task",
```

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

上一行过长后的续行，保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 main，把可复用的训练、评估或数据处理步骤独立出来。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

写出结果表、图或日志，供论文图表和复查使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

上一行过长后的续行，保持同一条代码的剩余内容。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

按验证指标选择超参数；无验证集时使用预声明固定参数。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

指定输入特征表、原始数据目录或输出表格/图像位置。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

MLP/基线训练流程中的普通语句，维持数据流或控制流。

Chapter 1 - MLP baseline and shared ML training pipeline

D:\thesis\code\training\run_manifest_da_baseline_splits.py

916	"model",	MLP/基线训练流程中的普通语句，维持数据流或控制流。
917	"best_params",	MLP/基线训练流程中的普通语句，维持数据流或控制流。
918	"train_accuracy",	MLP/基线训练流程中的普通语句，维持数据流或控制流。
919	"validation_accuracy",	MLP/基线训练流程中的普通语句，维持数据流或控制流。
920	"test_accuracy",	MLP/基线训练流程中的普通语句，维持数据流或控制流。
921	"train_balanced_accuracy",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
922	"validation_balanced_accuracy",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
923	"test_balanced_accuracy",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
924	"train_macro_f1",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
925	"validation_macro_f1",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
926	"test_macro_f1",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
927	]	MLP/基线训练流程中的普通语句，维持数据流或控制流。
928	print("Splits")	写出结果表、图或日志，供论文图表和复查使用。
929	print(splits.to_string(index=False))	写出结果表、图或日志，供论文图表和复查使用。
930	print("\nResults")	写出结果表、图或日志，供论文图表和复查使用。
931	print(results[cols].to_string(index=False))	写出结果表、图或日志，供论文图表和复查使用。
932		空行，用来分隔逻辑块，保持代码结构可读。
933		空行，用来分隔逻辑块，保持代码结构可读。
934	if __name__ == "__main__":	条件分支，按数据可用性、划分策略或方法变体选择严格路径。
935	main()	MLP/基线训练流程中的普通语句，维持数据流或控制流。

Chapter 2 - DANN training

This chapter contains the DANN feature extractor G_f , label classifier G_y , domain classifier G_d , gradient reversal, split construction, training loop, and model selection.

Source file: D:\thesis\code\training\run_dann_multi_splits.py. Total lines: 765. The following pages preserve the source order and line numbers.

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
1 from __future__ import annotations
2
3 import re
4 from pathlib import Path
5
6 import numpy as np
7 import pandas as pd
8 import torch
9 import torch.nn as nn
10 import torch.nn.functional as F
11
12
13 OUT_ROOT = Path(r"D:\thesis")
14 TABLE_DIR = OUT_ROOT / "tables"
15 FIG_DIR = OUT_ROOT / "figures" / "dann_learning"
16 FEATURE_PATH = TABLE_DIR / "manifest_baseline_as_air_features.csv"
17
18 TASKS = [
19     ("three_class_with_air", ["air", "alcohol", "acetone"]),
20 ]
21 REPORT_TASKS = [
22     ("three_class_with_air", ["air", "alcohol", "acetone"]),
23 ]
24
25 LAMBDA_VALUES = [0.05, 0.2, 0.5]
26 SEEDS = [7, 11, 23]
27 DEFAULT_LAMBDA = 0.2
28 DEFAULT_SEED = 7
29 NO_VALIDATION_SELECTION = "no validation; fixed hyperparameters"
30 EPOCHS = 400
31 LEARNING_RATE = 0.01
32 WEIGHT_DECAY = 1e-4
33
34
35 class GradientReverse(torch.autograd.Function):
36     """Gradient reversal layer used by standard DANN/RevGrad implementations."""
37     @staticmethod
38     def forward(ctx, x: torch.Tensor, lambd: float) -> torch.Tensor:
39         ctx.lambd = lambd
40         return x.view_as(x)
41
42     @staticmethod
43     def backward(ctx, grad_output: torch.Tensor) -> tuple[torch.Tensor, None]:
44         return -ctx.lambd * grad_output, None
45
46
47 def grad_reverse(x: torch.Tensor, lambd: float) -> torch.Tensor:
48     return GradientReverse.apply(x, lambd)
49
50
51 class FeatureExtractor(nn.Module):
52     def __init__(self, n_features: int) -> None:
53         super().__init__()
54         self.net = nn.Sequential(
55             nn.Linear(n_features, 32),
56             nn.ReLU(),
57             nn.Dropout(0.05),
58             nn.Linear(32, 16),
```

启用新版 Python 注解行为，避免类型注解在运行时过早求值。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义实验常量、路径、候选超参数或固定配置。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义类 GradientReverse，封装该方法的结构或状态。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义函数 forward，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义函数 backward，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

grad_reverse，把可复用的训练、评估或数据处理步骤独立出来。

实现梯度反转层，使特征提取器学习更难区分 source/target 的表示。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义类 FeatureExtractor，封装该方法的结构或状态。

定义函数 init，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
59         nn.ReLU(),
60     )
61
62     def forward(self, x: torch.Tensor) -> torch.Tensor:
63         return self.net(x)
64
65
66 class LabelClassifier(nn.Module):
67     def __init__(self, n_classes: int) -> None:
68         super().__init__()
69         self.net = nn.Linear(16, n_classes)
70
71     def forward(self, h: torch.Tensor) -> torch.Tensor:
72         return self.net(h)
73
74
75 class DomainClassifier(nn.Module):
76     def __init__(self) -> None:
77         super().__init__()
78         self.net = nn.Sequential(
79             nn.Linear(16, 16),
80             nn.ReLU(),
81             nn.Linear(16, 2),
82         )
83
84     def forward(self, h: torch.Tensor) -> torch.Tensor:
85         return self.net(h)
86
87
88 class DANN(nn.Module):
89     """DANN structure: feature extractor + label classifier + domain classifier."""
90
91     def __init__(self, n_features: int, n_classes: int) -> None:
92         super().__init__()
93         self.feature_extractor = FeatureExtractor(n_features)
94         self.label_classifier = LabelClassifier(n_classes)
95         self.domain_classifier = DomainClassifier()
96
97         # Backward-compatible aliases used by reporting scripts.
98         self.feature = self.feature_extractor
99         self.classifier = self.label_classifier
100        self.domain = self.domain_classifier
101
102    def forward_class(self, x: torch.Tensor) -> torch.Tensor:
103        h = self.feature_extractor(x)
104        return self.label_classifier(h)
105
106    def forward_domain(self, x: torch.Tensor, lambd: float) -> torch.Tensor:
107        h = self.feature_extractor(x)
108        return self.domain_classifier(grad_reverse(h, lambd))
109
110    def domain_no_grl(self, x: torch.Tensor) -> torch.Tensor:
111        return self.domain_classifier(self.feature_extractor(x))
112
113
114 def feature_columns(df: pd.DataFrame) -> list[str]:
115     return [c for c in df.columns if c[2] in {f"s{i}" for i in range(1, 7)} and c[2:3] == "_"]
116
```

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 'forward'，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义类 'LabelClassifier'，封装该方法的结构或状态。

定义函数 '__init__'，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 'forward'，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义类 'DomainClassifier'，封装该方法的结构或状态。

定义函数 '__init__'，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 'forward'，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义类 'DANN'，封装该方法的结构或状态。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 '__init__'，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

构造或使用特征提取器 G_f，将输入特征映射到潜在表示 h。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

空行，用来分隔逻辑块，保持代码结构可读。

代码作者的注释，说明下面代码块的实验意图或方法边界。

构造或使用特征提取器 G_f，将输入特征映射到潜在表示 h。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

空行，用来分隔逻辑块，保持代码结构可读。

forward_class，把可复用的训练、评估或数据处理步骤独立出来。

构造或使用特征提取器 G_f，将输入特征映射到潜在表示 h。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

空行，用来分隔逻辑块，保持代码结构可读。

forward_domain，把可复用的训练、评估或数据处理步骤独立出来。

构造或使用特征提取器 G_f，将输入特征映射到潜在表示 h。

实现梯度反转层，使特征提取器学习更难区分 source/target 的表示。

空行，用来分隔逻辑块，保持代码结构可读。

domain_no_grl，把可复用的训练、评估或数据处理步骤独立出来。

构造或使用特征提取器 G_f，将输入特征映射到潜在表示 h。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

feature_columns，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
117
118 def encode(labels: np.ndarray, classes: list[str]) -> np.ndarray:
119     mapping = {label: i for i, label in enumerate(classes)}
120     return np.array([mapping[str(v)] for v in labels], dtype=np.int64)
121
122
123 def metric_dict(y_true: np.ndarray, y_pred: np.ndarray, n_classes: int) -> dict[str, float]:
124     if len(y_true) == 0:
125         raise ValueError("Cannot compute metrics on an empty evaluation set.")
126     cm = np.zeros((n_classes, n_classes), dtype=int)
127     for t, p in zip(y_true, y_pred):
128         cm[int(t), int(p)] += 1
129     recalls, f1s = [], []
130     for c in range(n_classes):
131         tp = cm[c, c]
132         precision = tp / max(1, cm[:, c].sum())
133         recall = tp / max(1, cm[c, :].sum())
134         recalls.append(recall)
135         f1s.append(2 * precision * recall / max(1e-12, precision + recall))
136     return {
137         "accuracy": float((y_true == y_pred).mean()),
138         "balanced_accuracy": float(np.mean(recalls)),
139         "macro_f1": float(np.mean(f1s)),
140     }
141
142
143 def class_counts(df: pd.DataFrame, classes: list[str]) -> str:
144     if len(df) == 0:
145         return ""
146     counts = df["label"].value_counts().reindex(classes, fill_value=0)
147     return "; ".join(f"{label}:{int(counts[label])}" for label in classes)
148
149
150 def days_text(df: pd.DataFrame) -> str:
151     if len(df) == 0:
152         return ""
153     chunks = []
154     for period in ["Period_I", "Period_II", "Period_III"]:
155         sub = df[df["period"].eq(period)]
156         if sub.empty:
157             continue
158         days = sorted(int(d) for d in sub["day_num"].dropna().unique() if int(d) > 0)
159         chunks.append(f"{period} Day{min(days)}-Day{max(days)}" if days else period)
160     return "; ".join(chunks)
161
162
163 def natural_key(value: str) -> tuple:
164     parts = re.split(r"(\d+)", str(value))
165     return tuple(int(part) if part.isdigit() else part for part in parts)
166
167
168 def period_i_holdout(df: pd.DataFrame, partition: str, scheme: str = "25_5") -> pd.Series:
169     mask = pd.Series(False, index=df.index)
170     period_i = df[df["period"].eq("Period_I")]
171     if scheme == "25_5":
172         validation_counts = {"air": 1, "alcohol": 2, "acetone": 2} if "air" in set(period_i["label"]) else {"alcohol": 2, "acetone": 2}
173     elif scheme == "20_10":
174         validation_counts = {"air": 2, "alcohol": 4, "acetone": 4} if "air" in set(period_i["label"]) else {"alcohol": 4, "acetone": 4}
```

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 encode，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

metric dict，把可复用的训练、评估或数据处理步骤独立出来。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

返回本函数计算出的模型、指标、表格行或中间结果。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

class counts，把可复用的训练、评估或数据处理步骤独立出来。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

返回本函数计算出的模型、指标、表格行或中间结果。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 days_text，把可复用的训练、评估或数据处理步骤独立出来。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

返回本函数计算出的模型、指标、表格行或中间结果。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

natural_key，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

period i holdout，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

D:\thesis\code\training\run_dann_multi_splits.py

```

175: else:
176:     raise ValueError(f"Unknown Period I holdout scheme: {scheme}")
177: for label, sub in period_i.groupby("label"):
178:     seed = 42 + sum(ord(ch) for ch in str(label))
179:     rng = np.random.default_rng(seed)
180:     ordered = sorted(sub.index, key=lambda idx: natural_key(df.loc[idx, "target_file"]))
181:     ordered = list(rng.permutation(ordered))
182:     val_n = min(validation_counts.get(str(label), 0), max(0, len(ordered) - 1))
183:     val_idx = set(ordered[:val_n])
184:     chosen = [idx for idx in ordered if idx not in val_idx] if partition == "source_train" else list(val_idx)
185:     mask.loc[chosen] = True
186: return mask
187
188
189 def target_domain_features(df: pd.DataFrame) -> pd.Series:
190     return df["period"].isin(["Period_II", "Period_III"])
191
192
193 SPLITS = [
194     {
195         "split": "S1_train_I_test_all",
196         "description": "Train Period I only; no validation; test all Period II/III days. UDA variants use Period II/III target features  
... without gas labels for domain loss.",
197         "source_train": lambda df: df["period"].eq("Period_I"),
198         "source_val": lambda df: pd.Series(False, index=df.index),
199         "target_labeled_train": lambda df: pd.Series(False, index=df.index),
200         "target_domain": target_domain_features,
201         "target_val": lambda df: pd.Series(False, index=df.index),
202         "target_test": lambda df: df["period"].isin(["Period_II", "Period_III"]),
203         "selection": NO_VALIDATION_SELECTION,
204     },
205     {
206         "split": "S2_train_I_internal_val_test_all",
207         "description": "Train a stratified Period I subset; validate remaining Period I; test all Period II/III days. Three-class uses  
... train 25 / validation 5.",
208         "source_train": lambda df: period_i_holdout(df, "source_train", "25_5"),
209         "source_val": lambda df: period_i_holdout(df, "source_val", "25_5"),
210         "target_labeled_train": lambda df: pd.Series(False, index=df.index),
211         "target_domain": target_domain_features,
212         "target_val": lambda df: pd.Series(False, index=df.index),
213         "target_test": lambda df: df["period"].isin(["Period_II", "Period_III"]),
214         "selection": "source_validation_labels",
215     },
216     {
217         "split": "S3_train_I_internal_20_10_test_all",
218         "description": "Train a stratified Period I subset; validate remaining Period I; test all Period II/III days. Three-class uses  
... train 20 / validation 10.",
219         "source_train": lambda df: period_i_holdout(df, "source_train", "20_10"),
220         "source_val": lambda df: period_i_holdout(df, "source_val", "20_10"),
221         "target_labeled_train": lambda df: pd.Series(False, index=df.index),
222         "target_domain": target_domain_features,
223         "target_val": lambda df: pd.Series(False, index=df.index),
224         "target_test": lambda df: df["period"].isin(["Period_II", "Period_III"]),
225         "selection": "source_validation_labels",
226     },
227     {
228         "split": "S4_train_I_val_D1_2",
229         "description": "Train Period I; validate Period II Day1-Day2; test Period II Day3-Day17 + Period III Day18-Day19.",

```

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义时间/域划分，明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

上一行过长的续行，保持同一条代码的剩余内容。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义时间/域划分，明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

上一行过长的续行，保持同一条代码的剩余内容。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义时间/域划分，明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
230     "source_train": lambda df: df["period"].eq("Period_I"),
231     "source_val": lambda df: pd.Series(False, index=df.index),
232     "target_labeled_train": lambda df: pd.Series(False, index=df.index),
233     "target_domain": target_domain_features,
234     "target_val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(1, 2),
235     "target_test": lambda df: (
236         (df["period"].eq("Period_II") & df["day_num"].between(3, 17))
237         | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
238     ),
239     "selection": "target_validation_labels",
240 },
241 {
242     "split": "S5_train_I_D1_val_D2_3",
243     "description": "Train Period I + Period II Day1; validate Period II Day2-Day3; test Period II Day4-Day17 + Period III
244         ... Day18-Day19.",
245     "source_train": lambda df: df["period"].eq("Period_I"),
246     "source_val": lambda df: pd.Series(False, index=df.index),
247     "target_labeled_train": lambda df: df["period"].eq("Period_II") & df["day_num"].eq(1),
248     "target_domain": target_domain_features,
249     "target_val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(2, 3),
250     "target_test": lambda df: (
251         (df["period"].eq("Period_II") & df["day_num"].between(4, 17))
252         | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
253     ),
254     "selection": "target_validation_labels",
255 },
256 {
257     "split": "S6_train_I_D1_2_val_D3_5",
258     "description": "Train Period I + Period II Day1-Day2; validate Period II Day3-Day5; test Period II Day6-Day17 + Period III
259         ... Day18-Day19.",
260     "source_train": lambda df: df["period"].eq("Period_I"),
261     "source_val": lambda df: pd.Series(False, index=df.index),
262     "target_labeled_train": lambda df: df["period"].eq("Period_II") & df["day_num"].between(1, 2),
263     "target_domain": target_domain_features,
264     "target_val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(3, 5),
265     "target_test": lambda df: (
266         (df["period"].eq("Period_II") & df["day_num"].between(6, 17))
267         | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
268     ),
269     "selection": "target_validation_labels",
270 },
271 {
272     "split": "S7_train_I_D1_3_val_D4_5",
273     "description": "Train Period I + Period II Day1-Day3; validate Period II Day4-Day5; test Period II Day6-Day17 + Period III
274         ... Day18-Day19.",
275     "source_train": lambda df: df["period"].eq("Period_I"),
276     "source_val": lambda df: pd.Series(False, index=df.index),
277     "target_labeled_train": lambda df: df["period"].eq("Period_II") & df["day_num"].between(1, 3),
278     "target_domain": target_domain_features,
279     "target_val": lambda df: df["period"].eq("Period_II") & df["day_num"].between(4, 5),
280     "target_test": lambda df: (
281         (df["period"].eq("Period_II") & df["day_num"].between(6, 17))
282         | (df["period"].eq("Period_III") & df["day_num"].between(18, 19))
283     ),
284     "selection": "target_validation_labels",
285 },
286 ]
```

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义时间/域划分，明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

上一行过长后的续行，保持同一条代码的剩余内容。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义时间/域划分，明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

上一行过长后的续行，保持同一条代码的剩余内容。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

定义时间/域划分，明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

上一行过长后的续行，保持同一条代码的剩余内容。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

D:\thesis\code\training\run_dann_multi_splits.py

空行, 用来分隔逻辑块, 保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

空行, 用来分隔逻辑块, 保持代码结构可读。

定义函数来分隔逻辑块, 保持代码结构可读。

standardize, 把可复用的训练、评估或数据处理步骤独立出来。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护: 输入、划分或特征不满足实验假设时直接停止。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护: 输入、划分或特征不满足实验假设时直接停止。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量, 供后续训练、评估或输出步骤使用。

显式失败保护: 输入、划分或特征不满足实验假设时直接停止。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行, 用来分隔逻辑块, 保持代码结构可读。

定义函数 to tensor, 把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行, 用来分隔逻辑块, 保持代码结构可读。

定义函数 to long, 把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行, 用来分隔逻辑块, 保持代码结构可读。

定义函数来分隔逻辑块, 保持代码结构可读。

class weights, 把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量, 供后续训练、评估或输出步骤使用。

显式失败保护: 输入、划分或特征不满足实验假设时直接停止。

保存中间变量, 供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

空行, 用来分隔逻辑块, 保持代码结构可读。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分, 明确训练、验证、测试样本来源。

返回本函数计算出的模型、指标、表格行或中间结果。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分, 明确训练、验证、测试样本来源。

返回本函数计算出的模型、指标、表格行或中间结果。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
343     if selection == NO_VALIDATION_SELECTION:
344         return None, None, NO_VALIDATION_SELECTION
345     raise ValueError(f"Unknown split selection policy: {selection}")
346
347
348 def predict(model: DANN, x: torch.Tensor) -> np.ndarray:
349     model.eval()
350     with torch.no_grad():
351         return model.forward_class(x).argmax(dim=1).cpu().numpy()
352
353
354 def predict_domain(model: DANN, x: torch.Tensor) -> np.ndarray:
355     model.eval()
356     with torch.no_grad():
357         return model.domain_no_gri(x).argmax(dim=1).cpu().numpy()
358
359
360 def train_model(
361     x_cls: np.ndarray,
362     y_cls: np.ndarray,
363     x_source_domain: np.ndarray,
364     x_target_domain: np.ndarray,
365     x_select: np.ndarray,
366     y_select: np.ndarray,
367     n_classes: int,
368     use_domain_loss: bool,
369     lambda_value: float,
370     seed: int,
371 ) -> tuple[DANN, dict[str, float]]:
372     torch.manual_seed(seed)
373     np.random.seed(seed)
374     model = DANN(x_cls.shape[1], n_classes)
375     optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE, weight_decay=WEIGHT_DECAY)
376
377     x_cls_t = to_tensor(x_cls)
378     y_cls_t = to_long(y_cls)
379     x_select_t = to_tensor(x_select) if x_select is not None else None
380     y_weight = class_weights(y_cls, n_classes)
381
382     if use_domain_loss and len(x_target_domain) == 0:
383         raise ValueError("Domain loss was requested, but x_target_domain is empty.")
384
385     x_dom_t = torch.empty((0, x_cls.shape[1]), dtype=torch.float32)
386     y_dom_t = torch.empty((0, ), dtype=torch.long)
387     if len(x_target_domain) > 0:
388         x_dom_t = torch.cat([to_tensor(x_source_domain), to_tensor(x_target_domain)], dim=0)
389         y_dom_t = torch.cat(
390             [
391                 torch.zeros(len(x_source_domain), dtype=torch.long),
392                 torch.ones(len(x_target_domain), dtype=torch.long),
393             ],
394             dim=0,
395         )
396
397     best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
398     best_score = (-1.0, -1.0)
399     best_epoch = 0
400     last_L = 0.0
```

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

返回本函数计算出的模型、指标、表格行或中间结果。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 predict，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

predict_domain，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

train_model，把可复用的训练、评估或数据处理步骤独立出来。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

设置随机种子，提高训练过程的可复现性。

设置随机种子，提高训练过程的可复现性。

保存中间变量，供后续训练、评估或输出步骤使用。

创建或调用优化器，更新神经网络参数。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

按验证指标选择超参数：无验证集时使用预声明固定参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
401 last_ld = 0.0
402 last_backward_loss_scalar = 0.0
403
404 for epoch in range(1, EPOCHS + 1):
405     model.train()
406     optimizer.zero_grad()
407     class_logits = model.forward_class(x_cls_t)
408     L = F.cross_entropy(class_logits, y_cls_t, weight=y_weight)
409     backward_loss_scalar = L
410     Ld = torch.tensor(0.0)
411     if use_domain_loss:
412         domain_logits = model.forward_domain(x_dom_t, lambda_value)
413         Ld = F.cross_entropy(domain_logits, y_dom_t)
414         backward_loss_scalar = L + Ld
415     backward_loss_scalar.backward()
416     optimizer.step()
417     last_L = float(L.detach().cpu())
418     last_ld = float(Ld.detach().cpu())
419     last_backward_loss_scalar = float(backward_loss_scalar.detach().cpu())
420
421     if x_select_t is not None and (epoch % 10 == 0 or epoch == EPOCHS):
422         select_pred = predict(model, x_select_t)
423         select_m = metric_dict(y_select, select_pred, n_classes)
424         score = (select_m["macro_f1"], select_m["balanced_accuracy"], select_m["accuracy"])
425         if score > best_score:
426             best_score = score
427             best_epoch = epoch
428             best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
429     elif x_select_t is None and epoch == EPOCHS:
430         best_epoch = epoch
431         best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
432
433 model.load_state_dict(best_state)
434 return model, {
435     "best_epoch": float(best_epoch),
436     "L_last": last_L,
437     "Ld_last": last_ld,
438     "backward_loss_scalar_last": last_backward_loss_scalar,
439     "class_loss_last": last_L,
440     "domain_loss_last": last_ld,
441 }
442
443
444 def evaluate_split(df: pd.DataFrame, split: dict, task_name: str, classes: list[str]) -> tuple[list[dict], list[dict]]:
445     task = df[df["label"].isin(classes)].copy().reset_index(drop=True)
446     masks = {
447         "source_train": split["source_train"](task),
448         "source_val": split["source_val"](task),
449         "target_labeled_train": split["target_labeled_train"](task),
450         "target_domain": split["target_domain"](task),
451         "target_val": split["target_val"](task),
452         "target_test": split["target_test"](task),
453     }
454     parts = {name: task[mask].copy() for name, mask in masks.items()}
455     if len(parts["source_train"]) == 0 or len(parts["target_test"]) == 0:
456         raise ValueError(f'"{split["split"]}" has empty source_train or target_test for {task_name}.')
457     if not set(classes).issubset(set(parts["source_train"]["label"])):
458         raise ValueError(f'"{split["split"]}" source_train is missing classes for {task_name}.')
```

保存中间变量，供后续训练、评估或输出步骤使用。

执行反向传播和参数更新。

空行，用来分隔逻辑块，保持代码结构可读。

循环遍历 split、模型、超参数、epoch 或样本记录。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

创建或调用优化器，更新神经网络参数。

保存中间变量，供后续训练、评估或输出步骤使用。

定义实验常量、路径、候选超参数或固定配置。

执行反向传播和参数更新。

计算损失项：分类损失、域损失或条件对齐损失。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

计算损失项：分类损失、域损失或条件对齐损失。

执行反向传播和参数更新。

执行反向传播和参数更新。

创建或调用优化器，更新神经网络参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

执行反向传播和参数更新。

空行，用来分隔逻辑块，保持代码结构可读。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

按验证指标选择超参数；无验证集时使用预先声明固定参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

返回本函数计算出的模型、指标、表格行或中间结果。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

执行反向传播和参数更新。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

evaluate_split，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分、明确训练、验证、测试样本来源。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分、明确训练、验证、测试样本来源。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
459     if len(parts["target_test"]) and not set(classes).issubset(set(parts["target_test"]["label"])):
460         raise ValueError(f'{split["split"]} target_test is missing classes for {task_name}.')
461
462     split_rows = []
463     usage_text = {
464         "source_train": "labels used for classifier loss",
465         "source_val": "labels used only for model selection",
466         "target_labeled_train": "labels used only by semi-supervised variants",
467         "target_domain": "gas labels not used; features used for UDA/domain loss",
468         "target_val": "labels used only for model selection",
469         "target_test": "labels used only for final evaluation",
470     }
471     for name, part in parts.items():
472         split_rows.append(
473             {
474                 "split": split["split"],
475                 "task": task_name,
476                 "partition": name,
477                 "n": len(part),
478                 "class_counts": class_counts(part, classes),
479                 "period_days": days_text(part),
480                 "label_usage": usage_text[name],
481             }
482         )
483
484     cols = feature_columns(task)
485     source_raw = parts["source_train"][cols].to_numpy(float)
486     source_val_raw = parts["source_val"][cols].to_numpy(float)
487     target_label_raw = parts["target_labeled_train"][cols].to_numpy(float)
488     target_domain_raw = parts["target_domain"][cols].to_numpy(float)
489     target_val_raw = parts["target_val"][cols].to_numpy(float)
490     target_test_raw = parts["target_test"][cols].to_numpy(float)
491
492     ref_arrays = [source_raw]
493     if len(target_domain_raw):
494         ref_arrays.append(target_domain_raw)
495     x_ref = np.vstack(ref_arrays)
496     source_x, source_val_x, target_label_x, target_domain_x, target_val_x, target_test_x = standardize(
497         x_ref,
498         source_raw,
499         source_val_raw,
500         target_label_raw,
501         target_domain_raw,
502         target_val_raw,
503         target_test_raw,
504     )
505
506     y_source = encode(parts["source_train"]["label"].to_numpy(str), classes)
507     y_source_val = encode(parts["source_val"]["label"].to_numpy(str), classes) if len(parts["source_val"]) else np.array([],
508         ... dtype=np.int64)
509     y_target_label = encode(parts["target_labeled_train"]["label"].to_numpy(str), classes) if len(parts["target_labeled_train"]) else
510         np.array([], dtype=np.int64)
511     y_target_val = encode(parts["target_val"]["label"].to_numpy(str), classes) if len(parts["target_val"]) else np.array([],
512         ... dtype=np.int64)
513     y_target_test = encode(parts["target_test"]["label"].to_numpy(str), classes)
514
515     x_select, y_select, selection_labels = select_validation_arrays(
516         split,
```

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分, 明确训练、验证、测试样本来源。

空行, 用来分隔逻辑块, 保持代码结构可读。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_y, 用气体标签训练分类目标。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

循环遍历 split、模型、超参数、epoch 或样本记录。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

定义时间/域划分, 明确训练、验证、测试样本来源。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

空行, 用来分隔逻辑块, 保持代码结构可读。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

空行, 用来分隔逻辑块, 保持代码结构可读。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

保存中间变量, 供后续训练、评估或输出步骤使用。

做标准化: 脚本中会检查无效或近常数特征, 避免静默错误。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

空行, 用来分隔逻辑块, 保持代码结构可读。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

上一行过长后的续行, 保持同一条代码的剩余内容。

保存中间变量, 供后续训练、评估或输出步骤使用。

上一行过长后的续行, 保持同一条代码的剩余内容。

保存中间变量, 供后续训练、评估或输出步骤使用。

上一行过长后的续行, 保持同一条代码的剩余内容。

保存中间变量, 供后续训练、评估或输出步骤使用。

空行, 用来分隔逻辑块, 保持代码结构可读。

保存中间变量, 供后续训练、评估或输出步骤使用。

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
514     source_val_x,
515     y_source_val,
516     target_val_x,
517     y_target_val,
518 )
519
520 rows = []
521 for variant in VARIANTS:
522     if variant["use_target_labels"] and len(target_label_x) == 0:
523         continue
524     if variant["use_target_labels"]:
525         x_cls = np.vstack([source_x, target_label_x])
526         y_cls = np.concatenate([y_source, y_target_label])
527         classifier_label_usage = "source labels + selected target labels"
528     else:
529         x_cls = source_x
530         y_cls = y_source
531         classifier_label_usage = "source labels only"
532
533 if selection_labels == NO_VALIDATION_SELECTION:
534     fixed_lambda = DEFAULT_LAMBDA if variant["use_domain_loss"] else 0.0
535     candidates = [(fixed_lambda, DEFAULT_SEED)]
536 else:
537     candidates = [(lambda_value, seed) for lambda_value in variant["lambdas"] for seed in SEEDS]
538
539 for lambda_value, seed in candidates:
540     model, info = train_model(
541         x_cls=x_cls,
542         y_cls=y_cls,
543         x_source_domain=source_x,
544         x_target_domain=target_domain_x,
545         x_select=x_select,
546         y_select=y_select,
547         n_classes=len(classes),
548         use_domain_loss=variant["use_domain_loss"],
549         lambda_value=lambda_value,
550         seed=seed,
551     )
552     train_m = metric_dict(y_cls, predict(model, to_tensor(x_cls)), len(classes))
553     select_m = (
554         {"accuracy": np.nan, "balanced_accuracy": np.nan, "macro_f1": np.nan}
555         if x_select is None
556         else metric_dict(y_select, predict(model, to_tensor(x_select)), len(classes))
557     )
558     test_m = metric_dict(y_target_test, predict(model, to_tensor(target_test_x)), len(classes))
559
560 domain_acc = np.nan
561 if len(target_domain_x) > 0:
562     x_dom = np.vstack([source_x, target_domain_x])
563     y_dom = np.concatenate([np.zeros(len(source_x), dtype=int), np.ones(len(target_domain_x), dtype=int)])
564     domain_pred = predict_domain(model, to_tensor(x_dom))
565     domain_acc = float((domain_pred == y_dom).mean())
566
567 rows.append(
568     {
569         "split": split["split"],
570         "description": split["description"],
571         "task": task_name,
```

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
构造或使用标签分类器 G_y，用气体标签训练分类目标。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
构造或使用标签分类器 G_y，用气体标签训练分类目标。
空行，用来分隔逻辑块，保持代码结构可读。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。
空行，用来分隔逻辑块，保持代码结构可读。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。
保存中间变量，供后续训练、评估或输出步骤使用。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
空行，用来分隔逻辑块，保持代码结构可读。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
定义时间/域划分，明确训练、验证、测试样本来源。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
572         "variant": variant["variant"],
573         "lambda": lambda_value,
574         "seed": seed,
575         "classifier_label_usage": classifier_label_usage,
576         "domain_label_usage": "source/target domain labels only; gas labels not used for target domain loss",
577         "model_selection_labels": selection_labels,
578         "best_epoch": int(info["best_epoch"]),
579         "L_last": info["L_last"],
580         "Ld_last": info["Ld_last"],
581         "backward_loss_scalar_last": info["backward_loss_scalar_last"],
582         "feature_extractor_objective": "min L - lambda*Ld via gradient reversal",
583         "class_loss_last": info["class_loss_last"],
584         "domain_loss_last": info["domain_loss_last"],
585         "classifier_train_n": len(y_cls),
586         "source_train_n": len(y_source),
587         "source_val_n": len(y_source_val),
588         "target_labeled_train_n": len(y_target_label),
589         "target_domain_n": len(target_domain_x),
590         "target_validation_n": len(y_target_val),
591         "target_test_n": len(y_target_test),
592         "train_accuracy": train_m["accuracy"],
593         "validation_accuracy": select_m["accuracy"],
594         "test_accuracy": test_m["accuracy"],
595         "train_balanced_accuracy": train_m["balanced_accuracy"],
596         "validation_balanced_accuracy": select_m["balanced_accuracy"],
597         "test_balanced_accuracy": test_m["balanced_accuracy"],
598         "train_macro_f1": train_m["macro_f1"],
599         "validation_macro_f1": select_m["macro_f1"],
600         "test_macro_f1": test_m["macro_f1"],
601         "train_test_accuracy_gap": train_m["accuracy"] - test_m["accuracy"],
602         "domain_discriminator_accuracy": domain_acc,
603     }
604 )
605 return rows, split_rows
606
607
608 def choose_best(results: pd.DataFrame) -> pd.DataFrame:
609     selected = []
610     for (_split, _task, _variant), sub in results.groupby(["split", "task", "variant"], sort=False):
611         no_validation = sub["model_selection_labels"].eq(NO_VALIDATION_SELECTION).all()
612         if no_validation:
613             default_lambda = 0.0 if _variant == "MLP source only" else DEFAULT_LAMBDA
614             fixed = sub[np.isclose(sub["lambda"].astype(float), default_lambda) & sub["seed"].eq(DEFAULT_SEED)]
615             if fixed.empty:
616                 raise ValueError(f"Missing fixed no-validation candidate for {_split} / {_task} / {_variant}.")
617             ordered = fixed
618         else:
619             ordered = sub.sort_values(
620                 ["validation_macro_f1", "validation_balanced_accuracy", "validation_accuracy", "lambda", "seed"],
621                 ascending=[False, False, False, True, True],
622             )
623         selected.append(ordered.iloc[0])
624     out = pd.DataFrame(selected).reset_index(drop=True)
625     no_validation = out["model_selection_labels"].eq(NO_VALIDATION_SELECTION)
626     out["selection_status"] = np.where(
627         no_validation,
628         "fixed hyperparameters because the split has no validation set",
629         "hyperparameters selected by validation macro-F1, balanced accuracy, and accuracy",

```

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
构造或使用标签分类器 G_y，用气体标签训练分类目标。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
执行反向传播和参数更新。
构造或使用特征提取器 G_L，将输入特征映射到潜在表示 h。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
构造或使用标签分类器 G_y，用气体标签训练分类目标。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
构造或使用域判别器，用 source/target 域标签计算对抗域损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
返回本函数计算出的模型、指标、表格行或中间结果。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
choose_best，把可复用的训练、评估或数据处理步骤独立出来。
保存中间变量，供后续训练、评估或输出步骤使用。
定义时间/域划分，明确训练、验证、测试样本来源。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
显式失败保护：输入、划分或特征不满足实验假设时直接停止。
保存中间变量，供后续训练、评估或输出步骤使用。
条件分支的备选路径，仍保持明确规则而不是隐式猜测。
保存中间变量，供后续训练、评估或输出步骤使用。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
保存中间变量，供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

D:\thesis\code\training\run_dann_multi_splits.py

```

630 )
631 out["judgment"] = out["selection_status"]
632 for col in ["validation_accuracy", "validation_balanced_accuracy", "validation_macro_f1"]:
633     out.loc[no_validation, col] = np.nan
634 return out
635
636
637 def fmt(v: float) -> str:
638     return "NA" if pd.isna(v) else f"{float(v):.3f}"
639
640
641 def short_split(name: str) -> str:
642     return name.split("_", 1)[0]
643
644
645 def write_svg(summary: pd.DataFrame, path: Path) -> None:
646     cols = [
647         ("Split", 54),
648         ("Task", 92),
649         ("Variant", 145),
650         ("Lam", 48),
651         ("Train", 58),
652         ("Val", 58),
653         ("Test", 58),
654         ("BA", 58),
655         ("F1", 58),
656         ("Gap", 58),
657         ("Dom", 58),
658         ("Label usage", 210),
659         ("Judgment", 245),
660     ]
661     rows = len(summary)
662     row_h = 31
663     width = 32 + sum(w for _, w in cols) + 32
664     height = 116 + row_h * (rows + 1) + 160
665     lines = [
666         f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width} {height}">',
667         '<rect width="100%" height="100%" fill="white"/>',
668         '<text x="24" y="32" font-family="Arial" font-size="22" font-weight="700">DANN Multi-split Results for E-nose Drift',
669         '... Adaptation</text>',
670         '<text x="24" y="56" font-family="Arial" font-size="13" fill="#555">Target gas labels are never used in DANN-UDA training. Semi',
671         '... variants use selected target-day labels for classifier calibration. Domain accuracy closer to 0.5 means more',
672         '... domain-invariant features.</text>',
673     ]
674     x0, y = 24, 84
675     x = x0
676     for label, w in cols:
677         lines.append(f'<rect x="{x}" y="{y}" width="{w}" height="{row_h}" fill="#f1f4f8" stroke="#d8dee8"/>')
678         lines.append(f'<text x="{x + 5}" y="{y + 20}" font-family="Arial" font-size="10" font-weight="700">{label}</text>')
679         x += w
680     y += row_h
681     for _, row in summary.iterrows():
682         vals = [
683             short_split(row["split"]),
684             "3-class",
685             row["variant"],
686             fmt(row["lambda"]),
687             fmt(row["train accuracy"]),
688         ]

```

DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
保存中间变量, 供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量, 供后续训练、评估或输出步骤使用。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 fmi, 把可复用的训练、评估或数据处理步骤独立出来。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 short split, 把可复用的训练、评估或数据处理步骤独立出来。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 write_sve, 把可复用的训练、评估或数据处理步骤独立出来。
保存中间变量, 供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
上一行过长的续行, 保持同一条代码的剩余内容。
构造或使用标签分类器 G_v, 用气体标签训练分类目标。
上一行过长的续行, 保持同一条代码的剩余内容。
上一行过长的续行, 保持同一条代码的剩余内容。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量, 供后续训练、评估或输出步骤使用。
定义时间/域划分, 明确训练、验证、测试样本来源。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句, 服务于分类损失和域对抗损失。

Chapter 2 - DANN training

D:\thesis\code\training\run_dann_multi_splits.py

```
685         fmt(row["validation_accuracy"]),
686         fmt(row["test_accuracy"]),
687         fmt(row["test_balanced_accuracy"]),
688         fmt(row["test_macro_f1"]),
689         fmt(row["train_test_accuracy_gap"]),
690         fmt(row["domain_discriminator_accuracy"]),
691         row["classifier_label_usage"],
692         row["judgment"],
693     ]
694     x = x0
695     for (label, w), val in zip(cols, vals):
696         fill = "white"
697         lines.append(f'<rect x="{x}" y="{y}" width="{w}" height="{row_h}" fill="{fill}" stroke="#d8dee8"/>')
698         lines.append(f'<text x="{x + 5}" y="{y + 20}" font-family="Arial" font-size="9">{str(val)[:38]}</text>')
699         x += w
700     y += row_h
701
702     legend = [
703         "S1: Period I labels train; no validation; all Period II/III labels used only for final evaluation.",
704         "S2: Period I train/validation holdout 25/5; all Period II/III labels used only for final evaluation.",
705         "S3: Period I train/validation holdout 20/10; all Period II/III labels used only for final evaluation.",
706         "S4: Period I train; Day1-2 validation; Day3-19 test.",
707         "S5: Period I + Day1 target labels train; Day2-3 validation; Day4-19 test.",
708         "S6: Period I + Day1-2 target labels train; Day3-5 validation; Day6-19 test.",
709         "S7: Period I + Day1-3 target labels train; Day4-5 validation; Day6-19 test.",
710         "UDA variants use Period II/III target features without gas labels for domain loss in every split.",
711     ]
712     y += 24
713     for line in legend:
714         lines.append(f'<text x="24" y="{y}" font-family="Arial" font-size="12" fill="#555">{line}</text>')
715         y += 18
716     lines.append("</svg>")
717     path.write_text("\n".join(lines), encoding="utf-8")
718
719
720 def main() -> None:
721     torch.set_num_threads(1)
722     TABLE_DIR.mkdir(parents=True, exist_ok=True)
723     FIG_DIR.mkdir(parents=True, exist_ok=True)
724     df = pd.read_csv(FEATURE_PATH)
725     all_rows, split_rows = [], []
726     for split in SPLITS:
727         for task_name, classes in REPORT_TASKS:
728             rows, split_info = evaluate_split(df, split, task_name, classes)
729             all_rows.extend(rows)
730             split_rows.extend(split_info)
731
732     results = pd.DataFrame(all_rows)
733     splits = pd.DataFrame(split_rows)
734     summary = choose_best(results)
735
736     results.to_csv(TABLE_DIR / "dann_multi_split_all_runs.csv", index=False)
737     summary.to_csv(TABLE_DIR / "dann_multi_split_selected_summary.csv", index=False)
738     splits.to_csv(TABLE_DIR / "dann_multi_split_partitions.csv", index=False)
739     write_svg(summary, FIG_DIR / "dann_multi_split_summary.svg")
740
741     print("PyTorch", torch.__version__, "CUDA", torch.cuda.is_available())
742     print("\nPartitions")
```

DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
构造或使用域判别器，用 source/target 域标签计算对抗域损失。
构造或使用标签分类器 G_y，用气体标签训练分类目标。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
保存中间变量，供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
保存中间变量，供后续训练、评估或输出步骤使用。
空行，用来分隔逻辑块，保持代码结构可读。
空行，用来分隔逻辑块，保持代码结构可读。
定义函数 main，把可复用的训练、评估或数据处理步骤独立出来。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
指定输入特征表、原始数据目录或输出表格/图像位置。
指定输入特征表、原始数据目录或输出表格/图像位置。
指定输入特征表、原始数据目录或输出表格/图像位置。
指定输入特征表、原始数据目录或输出表格/图像位置。
保存中间变量，供后续训练、评估或输出步骤使用。
定义时间/域划分，明确训练、验证、测试样本来源。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量，供后续训练、评估或输出步骤使用。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
空行，用来分隔逻辑块，保持代码结构可读。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
按验证指标选择超参数；无验证集时使用预声明固定参数。
空行，用来分隔逻辑块，保持代码结构可读。
指定输入特征表、原始数据目录或输出表格/图像位置。
指定输入特征表、原始数据目录或输出表格/图像位置。
指定输入特征表、原始数据目录或输出表格/图像位置。
指定输入特征表、原始数据目录或输出表格/图像位置。
空行，用来分隔逻辑块，保持代码结构可读。
写出结果表、图或日志，供论文图表和复查使用。
写出结果表、图或日志，供论文图表和复查使用。

743	print(splits.to_string(index=False))	写出结果表、图或日志，供论文图表和复查使用。
744	print("\nSelected summary")	写出结果表、图或日志，供论文图表和复查使用。
745	show_cols = [	保存中间变量，供后续训练、评估或输出步骤使用。
746	"split",	定义时间/域划分，明确训练、验证、测试样本来源。
747	"task",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
748	"variant",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
749	"lambda",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
750	"seed",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
751	"classifier_label_usage",	构造或使用标签分类器 G_v，用气体标签训练分类目标。
752	"model_selection_labels",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
753	"train_accuracy",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
754	"validation_accuracy",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
755	"test_accuracy",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
756	"test_balanced_accuracy",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不平衡。
757	"test_macro_f1",	计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不平衡。
758	"domain_discriminator_accuracy",	构造或使用域判别器，用 source/target 域标签计算对抗域损失。
759	"judgment",	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
760	]	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。
761	print(summary[show_cols].to_string(index=False))	写出结果表、图或日志，供论文图表和复查使用。
762		空行，用来分隔逻辑块，保持代码结构可读。
763		空行，用来分隔逻辑块，保持代码结构可读。
764	if __name__ == "__main__":	条件分支，按数据可用性、划分策略或方法变体选择严格路径。
765	main()	DANN 训练流程中的结构语句，服务于分类损失和域对抗损失。

Chapter 3 - Official CDAN training

This chapter contains official CDAN-style conditional adversarial training with $T(h,g)=g$ outer h and a conditional domain discriminator G_{cd} .

Source file: D:\thesis\code\training\run_cdan_multi_splits.py. Total lines: 414. The following pages preserve the source order and line numbers.

Chapter 3 - Official CDAN training

D:\thesis\code\training\run_cdan_multi_splits.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import numpy as np
6 import pandas as pd
7 import torch
8 import torch.nn as nn
9 import torch.nn.functional as F
10
11 import run_dann_multi_splits as base
12
13
14 OUT_ROOT = Path("D:\thesis")
15 TABLE_DIR = OUT_ROOT / "tables"
16 FIG_DIR = OUT_ROOT / "figures" / "dann_learning"
17 FEATURE_PATH = TABLE_DIR / "manifest_baseline_as_air_features.csv"
18
19 # Official CDAN-style conditional adversarial variants.
20 # This script is prepared for the thesis data but is not run automatically.
21 VARIANTS = [
22     {"variant": "CDAN-UDA", "use_target_labels": False, "lambdas": base.LAMBDA_VALUES},
23     {"variant": "CDAN-semi", "use_target_labels": True, "lambdas": base.LAMBDA_VALUES},
24 ]
25
26
27 class CDANDomainDiscriminator(nn.Module):
28     """Domain discriminator for official CDAN-style conditional features.
29
30     Instead of receiving h directly as in DANN, this discriminator receives
31     T(h, g) = g outer h, flattened to C*H dimensions.
32     """
33
34     def __init__(self, feature_dim: int, n_classes: int) -> None:
35         super().__init__()
36         cdan_dim = feature_dim * n_classes
37         self.net = nn.Sequential(
38             nn.Linear(cdan_dim, 32),
39             nn.ReLU(),
40             nn.Linear(32, 2),
41         )
42
43     def forward(self, conditioned: torch.Tensor) -> torch.Tensor:
44         return self.net(conditioned)
45
46
47 def multilinear_condition(h: torch.Tensor, logits: torch.Tensor, detach_predictions: bool = True) -> torch.Tensor:
48     """Build the CDAN multilinear map T(h, g).
49
50     Args:
51         h: latent feature h, shape (N, H), here H=16.
52         logits: gas classifier logits before softmax, shape (N, C).
53         detach_predictions: if True, stop the adversarial loss from updating
54             the classifier through g. This follows the common official CDAN
55             PyTorch implementation pattern.
56
57     Returns:
58         Flattened outer product g x h, shape (N, C*H).
```

启用新版 Python 注解行为，避免类型注解在运行时过早求值。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

代码作者的注释，说明下面代码块的实验意图或方法边界。

代码作者的注释，说明下面代码块的实验意图或方法边界。

定义实验常量、路径、候选超参数或固定配置。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义类 'CDANDomainDiscriminator'，封装该方法的结构或状态。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

做外积，形成条件域判别输入。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 'init'，把可复用的训练、评估或数据处理步骤独立出来。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 'forward'，把可复用的训练、评估或数据处理步骤独立出来。

做外积，形成条件域判别输入。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 'multilinear_condition'，把可复用的训练、评估或数据处理步骤独立出来。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

保存中间变量，供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_v，用气体标签训练分类目标。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

构造或使用标签分类器 G_v，用气体标签训练分类目标。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

做外积，形成条件域判别输入。

D:\thesis\code\training\run_cdan_multi_splits.py

CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
空行, 用来分隔逻辑块, 保持代码结构可读。
保存中间变量, 供后续训练、评估或输出步骤使用。
条件分支, 按数据可用性、划分策略或方法变体选择严格路径。
在训练域或条件域上提供训练、评估或输出步骤使用。
做外积, 形成条件域判别输入。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义类 CDAN, 封装该方法的结构或状态。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
空行, 用来分隔逻辑块, 保持代码结构可读。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 init, 把可复用的训练、评估或数据处理步骤独立出来。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。
构造或使用标签分类器 G_y, 用气体标签训练分类目标。
构造或使用域判别器, 用 source/target 域标签计算对抗损失。
保存中间变量, 供后续训练、评估或输出步骤使用。
构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。
构造或使用标签分类器 G_y, 用气体标签训练分类目标。
空行, 用来分隔逻辑块, 保持代码结构可读。
forward features, 把可复用的训练、评估或数据处理步骤独立出来。
构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。
空行, 用来分隔逻辑块, 保持代码结构可读。
forward class, 把可复用的训练、评估或数据处理步骤独立出来。
构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。
构造或使用标签分类器 G_y, 用气体标签训练分类目标。
空行, 用来分隔逻辑块, 保持代码结构可读。
forward domain, 把可复用的训练、评估或数据处理步骤独立出来。
构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。
构造或使用域判别器, 用 source/target 域标签计算对抗损失。
做外积, 形成条件域判别输入。
实现梯度反转层, 使特征提取器学习更难区分 source/target 的表示。
空行, 用来分隔逻辑块, 保持代码结构可读。
domain no_grad, 把可复用的训练、评估或数据处理步骤独立出来。
构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。
构造或使用域判别器, 用 source/target 域标签计算对抗损失。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 predict, 把可复用的训练、评估或数据处理步骤独立出来。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
predict domain, 把可复用的训练、评估或数据处理步骤独立出来。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
CDAN 训练流程中的结构语句, 服务于条件域对抗训练。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。

D:\thesis\code\training\run_cdan_multi_splits.py

```

111 def train_cdan_model(
112     x_cls: np.ndarray,
113     y_cls: np.ndarray,
114     x_source_domain: np.ndarray,
115     x_target_domain: np.ndarray,
116     x_select: np.ndarray | None,
117     y_select: np.ndarray | None,
118     n_classes: int,
119     lambda_value: float,
120     seed: int,
121 ) -> tuple[CDAN, dict[str, float]]:
122     torch.manual_seed(seed)
123     np.random.seed(seed)
124     model = CDAN(x_cls.shape[1], n_classes)
125     optimizer = torch.optim.Adam(model.parameters(), lr=base.LEARNING_RATE, weight_decay=base.WEIGHT_DECAY)
126
127     x_cls_t = base.to_tensor(x_cls)
128     y_cls_t = base.to_long(y_cls)
129     x_select_t = base.to_tensor(x_select) if x_select is not None else None
130     y_weight = base.class_weights(y_cls, n_classes)
131     if len(x_target_domain) == 0:
132         raise ValueError("CDAN domain loss was requested, but x_target_domain is empty.")
133
134     x_dom_t = torch.cat([base.to_tensor(x_source_domain), base.to_tensor(x_target_domain)], dim=0)
135     y_dom_t = torch.cat(
136         [
137             torch.zeros(len(x_source_domain), dtype=torch.long),
138             torch.ones(len(x_target_domain), dtype=torch.long),
139         ],
140         dim=0,
141     )
142
143     best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
144     best_score = (-1.0, -1.0, -1.0)
145     best_epoch = 0
146     last_L = 0.0
147     last_Ld = 0.0
148     last_backward_loss_scalar = 0.0
149
150     for epoch in range(1, base.EPOCHS + 1):
151         model.train()
152         optimizer.zero_grad()
153
154         class_logits = model.forward_class(x_cls_t)
155         L = F.cross_entropy(class_logits, y_cls_t, weight=y_weight)
156
157         domain_logits = model.forward_domain(x_dom_t, lambda_value)
158         Ld = F.cross_entropy(domain_logits, y_dom_t)
159
160         backward_loss_scalar = L + Ld
161         backward_loss_scalar.backward()
162         optimizer.step()
163
164         last_L = float(L.detach().cpu())
165         last_Ld = float(Ld.detach().cpu())
166         last_backward_loss_scalar = float(backward_loss_scalar.detach().cpu())
167
168         if x_select_t is not None and (epoch % 10 == 0 or epoch == base.EPOCHS):

```

train_cdan_mode。把可复用的训练、评估或数据预处理步骤独立出来。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

设置随机种子，提高训练过程的可复现性。

设置随机种子，提高训练过程的可复现性。

保存中间变量，供后续训练、评估或输出步骤使用。

创建或调用优化器，更新神经网络参数。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

按验证指标选择超参数；无验证集时使用预声明固定参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

执行反向传播和参数更新。

空行，用来分隔逻辑块，保持代码结构可读。

循环遍历 split、模型、超参数、epoch 或样本记录。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

创建或调用优化器，更新神经网络参数。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

计算损失项：分类损失、域损失或条件对齐损失。

空行，用来分隔逻辑块，保持代码结构可读。

执行反向传播和参数更新。

执行反向传播和参数更新。

创建或调用优化器，更新神经网络参数。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

执行反向传播和参数更新。

空行，用来分隔逻辑块，保持代码结构可读。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

Chapter 3 - Official CDAN training

D:\thesis\code\training\run_cdan_multi_splits.py

```
175     select_pred = predict(model, x_select_t)
176     select_m = base.metric_dict(y_select, select_pred, n_classes)
177     score = (select_m["macro_f1"], select_m["balanced_accuracy"], select_m["accuracy"])
178     if score > best_score:
179         best_score = score
180         best_epoch = epoch
181         best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
182     elif x_select_t is None and epoch == base.EPOCHS:
183         best_epoch = epoch
184         best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
185
186     model.load_state_dict(best_state)
187     return model, {
188         "best_epoch": float(best_epoch),
189         "t_last": last_L,
190         "ld_cdan_last": last_Ld,
191         "backward_loss_scalar_last": last_backward_loss_scalar,
192         "feature_extractor_objective": "min L - lambda*Ld_cdan via GRL on T(h,g)",
193     }
194
195
196 def evaluate_split(df: pd.DataFrame, split: dict, task_name: str, classes: list[str]) -> tuple[list[dict], list[dict]]:
197     task = df[df["label"].isin(classes)].copy().reset_index(drop=True)
198     masks = {
199         "source_train": split["source_train"](task),
200         "source_val": split["source_val"](task),
201         "target_labeled_train": split["target_labeled_train"](task),
202         "target_domain": split["target_domain"](task),
203         "target_val": split["target_val"](task),
204         "target_test": split["target_test"](task),
205     }
206     parts = {name: task[mask].copy() for name, mask in masks.items()}
207     if len(parts["source_train"]) == 0 or len(parts["target_domain"]) == 0 or len(parts["target_test"]) == 0:
208         raise ValueError(f"{split['split']} / {task_name} has empty source_train, target_domain, or target_test.")
209     if not set(classes).issubset(set(parts["source_train"]["label"])):
210         raise ValueError(f"{split['split']} / {task_name} source_train is missing required classes.")
211     if len(parts["target_test"]) and not set(classes).issubset(set(parts["target_test"]["label"])):
212         raise ValueError(f"{split['split']} / {task_name} target_test is missing required classes.")
213
214     split_rows = []
215     usage_text = {
216         "source_train": "gas labels used for classifier loss L",
217         "source_val": "labels used only for model selection",
218         "target_labeled_train": "gas labels used only by CDAN-semi",
219         "target_domain": "gas labels not used; features used for CDAN domain loss Ld_cdan",
220         "target_val": "labels used only for model selection",
221         "target_test": "labels used only for final evaluation",
222     }
223     for name, part in parts.items():
224         split_rows.append(
225             {
226                 "split": split["split"],
227                 "task": task_name,
228                 "partition": name,
229                 "n": len(part),
230                 "class_counts": base.class_counts(part, classes),
231                 "period_days": base.days_text(part),
232                 "label_usage": usage_text[name],
```

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

按验证指标选择超参数；无验证集时使用预声明固定参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

返回本函数计算出的模型、指标、表格行或中间结果。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

执行反向传播和参数更新。

构造或使用特征提取器 G_f，将输入特征映射到潜在表示 h。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

evaluate_split，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分，明确训练、验证、测试样本来源。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分，明确训练、验证、测试样本来源。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分，明确训练、验证、测试样本来源。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

循环遍历 split、模型、超参数、epoch 或样本记录。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

定义时间/域划分，明确训练、验证、测试样本来源。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

Chapter 3 - Official CDAN training

D:\thesis\code\training\run_cdan_multi_splits.py

```
233     }
234 )
235
236 cols = base.feature_columns(task)
237 source_raw = parts["source_train"][cols].to_numpy(float)
238 source_val_raw = parts["source_val"][cols].to_numpy(float)
239 target_label_raw = parts["target_labeled_train"][cols].to_numpy(float)
240 target_domain_raw = parts["target_domain"][cols].to_numpy(float)
241 target_val_raw = parts["target_val"][cols].to_numpy(float)
242 target_test_raw = parts["target_test"][cols].to_numpy(float)
243
244 x_ref = np.vstack([source_raw, target_domain_raw])
245 source_x, source_val_x, target_label_x, target_domain_x, target_val_x, target_test_x = base.standardize(
246     x_ref,
247     source_raw,
248     source_val_raw,
249     target_label_raw,
250     target_domain_raw,
251     target_val_raw,
252     target_test_raw,
253 )
254
255 y_source = base.encode(parts["source_train"]["label"].to_numpy(str), classes)
256 y_source_val = base.encode(parts["source_val"]["label"].to_numpy(str), classes) if len(parts["source_val"]) else np.array([],
257     ... dtype=np.int64)
258 y_target_label = base.encode(parts["target_labeled_train"]["label"].to_numpy(str), classes) if len(parts["target_labeled_train"])
259     ... else np.array([], dtype=np.int64)
260 y_target_val = base.encode(parts["target_val"]["label"].to_numpy(str), classes) if len(parts["target_val"]) else np.array([],
261     ... dtype=np.int64)
262 y_target_test = base.encode(parts["target_test"]["label"].to_numpy(str), classes)
263
264 x_select, y_select, selection_labels = base.select_validation_arrays(
265     split,
266     source_val_x,
267     y_source_val,
268     target_val_x,
269     y_target_val,
270 )
271
272 rows = []
273 for variant in VARIANTS:
274     if variant["use_target_labels"] and len(target_label_x) == 0:
275         continue
276     if variant["use_target_labels"]:
277         x_cls = np.vstack([source_x, target_label_x])
278         y_cls = np.concatenate([y_source, y_target_label])
279         classifier_label_usage = "source labels + selected target labels"
280     else:
281         x_cls = source_x
282         y_cls = y_source
283         classifier_label_usage = "source labels only"
284
285     if selection_labels == base.NO_VALIDATION_SELECTION:
286         candidates = [(base.DEFAULT_LAMBDA, base.DEFAULT_SEED)]
287     else:
288         candidates = [(lambda_value, seed) for lambda_value in variant["lambdas"] for seed in base.SEEDS]
289
290     for lambda_value, seed in candidates:
```

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

做标准化；脚本中会检查无效或近常数特征，避免静默错误。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

上一行过长后的续行，保持同一条代码的剩余内容。

保存中间变量，供后续训练、评估或输出步骤使用。

上一行过长后的续行，保持同一条代码的剩余内容。

保存中间变量，供后续训练、评估或输出步骤使用。

上一行过长后的续行，保持同一条代码的剩余内容。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_v，用气体标签训练分类目标。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_v，用气体标签训练分类目标。

空行，用来分隔逻辑块，保持代码结构可读。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

循环遍历 split、模型、超参数、epoch 或样本记录。

Chapter 3 - Official CDAN training

D:\thesis\code\training\run_cdan_multi_splits.py

```
288     model, info = train_cdan_model(
289         x_cls=x_cls,
290         y_cls=y_cls,
291         x_source_domain=source_x,
292         x_target_domain=target_domain_x,
293         x_select=x_select,
294         y_select=y_select,
295         n_classes=len(classes),
296         lambda_value=lambda_value,
297         seed=seed,
298     )
299     train_m = base.metric_dict(y_cls, predict(model, base.to_tensor(x_cls)), len(classes))
300     select_m = (
301         {"accuracy": np.nan, "balanced_accuracy": np.nan, "macro_f1": np.nan}
302         if x_select is None
303         else base.metric_dict(y_select, predict(model, base.to_tensor(x_select)), len(classes))
304     )
305     test_m = base.metric_dict(y_target_test, predict(model, base.to_tensor(target_test_x)), len(classes))
306
307     x_dom = np.vstack([source_x, target_domain_x])
308     y_dom = np.concatenate([np.zeros(len(source_x), dtype=int), np.ones(len(target_domain_x), dtype=int)])
309     domain_pred = predict_domain(model, base.to_tensor(x_dom))
310     domain_acc = float((domain_pred == y_dom).mean())
311
312     rows.append(
313         {
314             "split": split["split"],
315             "description": split["description"],
316             "task": task_name,
317             "variant": variant["variant"],
318             "lambda": lambda_value,
319             "seed": seed,
320             "classifier_label_usage": classifier_label_usage,
321             "domain_label_usage": "source/target domain labels only; gas labels not used for CDAN domain loss",
322             "model_selection_labels": selection_labels,
323             "best_epoch": int(info["best_epoch"]),
324             "L_last": info["L_last"],
325             "Ld_cdan_last": info["Ld_cdan_last"],
326             "backward_loss_scalar_last": info["backward_loss_scalar_last"],
327             "feature_extractor_objective": info["feature_extractor_objective"],
328             "condition_map": "T(h,g)=softmax(G_y(h)) outer h; shape N x (C*16)",
329             "classifier_train_n": len(y_cls),
330             "source_train_n": len(y_source),
331             "source_val_n": len(y_source_val),
332             "target_labeled_train_n": len(y_target_label),
333             "target_domain_n": len(target_domain_x),
334             "target_validation_n": len(y_target_val),
335             "target_test_n": len(y_target_test),
336             "train_accuracy": train_m["accuracy"],
337             "validation_accuracy": select_m["accuracy"],
338             "test_accuracy": test_m["accuracy"],
339             "train_balanced_accuracy": train_m["balanced_accuracy"],
340             "validation_balanced_accuracy": select_m["balanced_accuracy"],
341             "test_balanced_accuracy": test_m["balanced_accuracy"],
342             "train_macro_f1": train_m["macro_f1"],
343             "validation_macro_f1": select_m["macro_f1"],
344             "test_macro_f1": test_m["macro_f1"],
345             "train_test_accuracy_gap": train_m["accuracy"] - test_m["accuracy"],
```

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

条件分支，按数据可用性，划分策略或方法变体选择严格路径。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

定义时间/域划分，明确训练、验证、测试样本来源。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

执行反向传播和参数更新。

构造或使用条件域判别器 G_h，用气体标签训练分类目标。

做外积，形成条件域判别输入。

构造或使用标签分类器 G_y，用气体标签训练分类目标。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不平衡。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

Chapter 3 - Official CDAN training

D:\thesis\code\training\run_cdan_multi_splits.py

```
346         "domain_discriminator_accuracy": domain_acc,
347     }
348 )
349 return rows, split_rows
350
351
352 def choose_best(results: pd.DataFrame) -> pd.DataFrame:
353     selected = []
354     for (_split, _task, _variant), sub in results.groupby(["split", "task", "variant"], sort=False):
355         no_validation = sub["model_selection_labels"].eq(base.NO_VALIDATION_SELECTION).all()
356         if no_validation:
357             fixed = sub[np.isclose(sub["lambda"].astype(float), base.DEFAULT_LAMBDA) & sub["seed"].eq(base.DEFAULT_SEED)]
358             if fixed.empty:
359                 raise ValueError(f"Missing fixed no-validation candidate for {_split} / {_task} / {_variant}.")
360             ordered = fixed
361         else:
362             ordered = sub.sort_values(
363                 ["validation_macro_f1", "validation_balanced_accuracy", "validation_accuracy", "lambda", "seed"],
364                 ascending=[False, False, False, True, True],
365             )
366             selected.append(ordered.iloc[0])
367     out = pd.DataFrame(selected).reset_index(drop=True)
368     no_validation = out["model_selection_labels"].eq(base.NO_VALIDATION_SELECTION)
369     out.loc[no_validation, ["validation_accuracy", "validation_balanced_accuracy", "validation_macro_f1"]] = np.nan
370     out["selection_status"] = np.where(
371         no_validation,
372         "fixed hyperparameters because the split has no validation set",
373         "hyperparameters selected by validation macro-F1, balanced accuracy, and accuracy",
374     )
375     best = out.groupby(["split", "task"])["test_accuracy"].transform("max")
376     out["judgment"] = out["selection_status"]
377     out.loc[np.isclose(out["test_accuracy"], best), "judgment"] += "; highest selected CDAN test accuracy in split"
378     out.loc[out["train_test_accuracy_gap"] > 0.25, "judgment"] += "; high train-test gap"
379     out.loc[out["domain_discriminator_accuracy"].between(0.45, 0.60, inclusive="both"), "judgment"] += "; domain near-invariant"
380     out.loc[out["domain_discriminator_accuracy"] > 0.75, "judgment"] += "; domain still separable"
381     return out
382
383
384 def main() -> None:
385     """Train CDAN variants and write CSV/SVG reports.
386
387     This function is intentionally not run during code generation. Execute this
388     script manually only when the machine has enough power/time for training.
389     """
390
391     torch.set_num_threads(1)
392     df = pd.read_csv(FEATURE_PATH)
393     all_rows: list[dict] = []
394     split_rows: list[dict] = []
395     for split in base.SPLITS:
396         for task_name, classes in base.TASKS:
397             rows, parts = evaluate_split(df, split, task_name, classes)
398             all_rows.extend(rows)
399             split_rows.extend(parts)
400
401     results = pd.DataFrame(all_rows)
402     splits = pd.DataFrame(split_rows)
403     summary = choose_best(results)
```

构造或使用域判别器，用 source/target 域标签计算对抗域损失。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

选择函数，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

显式失败保护：输入、划分或特征不满足实验假设时直接停止。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的备选路径，仍保持明确规则而不是隐式猜测。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

定义时间/域划分，明确训练、验证、测试样本来源。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

构造或使用域判别器，用 source/target 域标签计算对抗域损失。

构造或使用域判别器，用 source/target 域标签计算对抗域损失。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 main，把可复用的训练、评估或数据处理步骤独立出来。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

指定输入特征表、原始数据目录或输出表格/图像位置。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

CDAN 训练流程中的结构语句，服务于条件域对抗训练。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

按验证指标选择超参数；无验证集时使用预声明固定参数。

Chapter 3 - Official CDAN training

D:\thesis\code\training\run_cdan_multi_splits.py

404		空行，用来分隔逻辑块，保持代码结构可读。
405	TABLE_DIR.mkdir(parents=True, exist_ok=True)	指定输入特征表、原始数据目录或输出表格/图像位置。
406	FIG_DIR.mkdir(parents=True, exist_ok=True)	指定输入特征表、原始数据目录或输出表格/图像位置。
407	results.to_csv(TABLE_DIR / "cdan_multi_split_all_runs.csv", index=False)	指定输入特征表、原始数据目录或输出表格/图像位置。
408	summary.to_csv(TABLE_DIR / "cdan_multi_split_selected_summary.csv", index=False)	指定输入特征表、原始数据目录或输出表格/图像位置。
409	splits.to_csv(TABLE_DIR / "cdan_multi_split_partitions.csv", index=False)	指定输入特征表、原始数据目录或输出表格/图像位置。
410	base.write_svg(summary, FIG_DIR / "cdan_multi_split_summary.svg")	指定输入特征表、原始数据目录或输出表格/图像位置。
411		空行，用来分隔逻辑块，保持代码结构可读。
412		空行，用来分隔逻辑块，保持代码结构可读。
413	if __name__ == "__main__":	条件分支，按数据可用性、划分策略或方法变体选择严格路径。
414	main()	CDAN 训练流程中的结构语句，服务于条件域对抗训练。

Chapter 4 - Custom C-DANN centroid alignment

This chapter contains the custom C-DANN variant that adds class-wise centroid alignment L_c to the DANN objective. It is intentionally separated from official CDAN.

Source file: D:\thesis\code\training\build_dann_conditional_report.py. Total lines: 573. The following pages preserve the source order and line numbers.

D:\thesis\code\training\build_dann_conditional_report.py

启用新版 Python 注解行为，避免类型注解在运行时过早求值。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

空行，用来分隔逻辑块，保持代码结构可读。

导入本脚本需要的库或项目内模块。

导入本脚本需要的库或项目内模块。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

定义实验常量、路径、候选超参数或固定配置。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

定义函数 one_hot，把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

将仅两期来分筛选逻辑，保持代码结构可读。

weighted_mean，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支的嵌套使用，用于处理复杂逻辑或距离约束。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

将仅两期来分筛选逻辑，保持代码结构可读。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

D:\thesis\code\training\build_dann_conditional_report.py

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

构造或使用标签分类器 G_{id}，用气体标签训练分类目标。

C-DANN 的条件条件 centroid alignment，用源域/目标域同类中心距离约束。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

空行两格来分隔逻辑块，保持代码结构可读。

train_conditional，把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

设置随机种子，提高训练过程的可复现性。

设置随机种子，提高训练过程的可复现性。

空行，用来分隔逻辑块，保持代码结构可读。

构造或使用标签分类器 G_v，用气体标签训练分类目标。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

C-DANN 的条件条件 centroid alignment，用源域/目标域同类中心距离约束。

C-DANN 的条件条件 centroid alignment，用源域/目标域同类中心距离约束。

保存中间变量，供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_v，用气体标签训练分类目标。

条件分支，按数据可用性、划分策略或方法变体选择严格路径。

构造或使用标签分类器 G_{id}，用气体标签训练分类目标。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

创建或调用优化器，更新神经网络参数。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

C-DANN 的条件条件 centroid alignment，用源域/目标域同类中心距离约束。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

按验证指标选择超参数；无验证集时使用预声明固定参数。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

Chapter 4 - Custom C-DANN centroid alignment

D:\thesis\code\training\build_dann_conditional_report.py

```
117     for epoch in range(1, dann.EPOCHS + 1):
118         model.train()
119         optimizer.zero_grad()
120         class_logits = model.forward_class(x_cls_t)
121         L = F.cross_entropy(class_logits, y_cls_t, weight=y_weight)
122         domain_logits = model.forward_domain(x_dom_t, lambda_value)
123         Ld = F.cross_entropy(domain_logits, y_dom_t)
124         Lc = conditional_alignment_loss(model, x_source_t, y_source, x_centroid_target_t, target_probs, len(classes))
125         backward_loss_scalar = L + Ld + align_strength * Lc
126         backward_loss_scalar.backward()
127         optimizer.step()
128
129     if epoch % LOG EVERY == 0 or epoch == 1 or epoch == dann.EPOCHS:
130         train_m = dann.metric_dict(y_cls, base.predict(model, x_cls), len(classes))
131         select_m = (
132             {"accuracy": np.nan, "balanced_accuracy": np.nan, "macro_f1": np.nan}
133             if x_select is None
134             else dann.metric_dict(y_select, base.predict(model, x_select), len(classes))
135         )
136         test_m = dann.metric_dict(y_test, base.predict(model, x_test), len(classes))
137         dom_acc = base.domain_accuracy(model, x_source, x_target)
138         curve = {
139             "epoch": epoch,
140             "L": float(L.detach()).cpu(),
141             "Ld": float(Ld.detach()).cpu(),
142             "Lc": float(Lc.detach()).cpu(),
143             "backward_loss_scalar": float(backward_loss_scalar.detach()).cpu(),
144             "class_loss": float(L.detach()).cpu(),
145             "domain_loss": float(Ld.detach()).cpu(),
146             "alignment_loss": float(Lc.detach()).cpu(),
147             "total_loss": float(backward_loss_scalar.detach()).cpu(),
148             "train_accuracy": train_m["accuracy"],
149             "validation_accuracy": select_m["accuracy"],
150             "test_accuracy": test_m["accuracy"],
151             "domain_discriminator_accuracy": dom_acc,
152         }
153         curves.append(curve)
154         mixing_proxy = -abs(dom_acc - 0.5) if np.isfinite(dom_acc) else -1.0
155         score = (select_m["macro_f1"], select_m["balanced_accuracy"], select_m["accuracy"])
156         if x_select is not None and score > best_score:
157             best_score = score
158             best_epoch = epoch
159             best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
160             last_info = {
161                 "train_accuracy": train_m["accuracy"],
162                 "validation_accuracy": select_m["accuracy"],
163                 "test_accuracy": test_m["accuracy"],
164                 "train_balanced_accuracy": train_m["balanced_accuracy"],
165                 "validation_balanced_accuracy": select_m["balanced_accuracy"],
166                 "test_balanced_accuracy": test_m["balanced_accuracy"],
167                 "train_macro_f1": train_m["macro_f1"],
168                 "validation_macro_f1": select_m["macro_f1"],
169                 "test_macro_f1": test_m["macro_f1"],
170                 "domain_discriminator_accuracy": dom_acc,
171                 "L_last": curve["L"],
172                 "Ld_last": curve["Ld"],
173                 "Lc_last": curve["Lc"],
174                 "backward_loss_scalar_last": curve["backward_loss_scalar"],
```

循环遍历 split、模型、超参数、epoch 或样本记录。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
创建或调用优化器，更新神经网络参数。
保存中间变量，供后续训练、评估或输出步骤使用。
定义实验常量、路径、候选超参数或固定配置。
保存中间变量，供后续训练、评估或输出步骤使用。
计算损失函数分类损失、域损失或条件熵损失。
C-DANN 的类条件 centroid alignment，用源域/目标域同类中心距离约束 h。
执行反向传播和参数更新。
创建或调用优化器，更新神经网络参数。
空行，用来分隔逻辑块，保持代码结构可读。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
保存中间变量，供后续训练、评估或输出步骤使用。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
构造或使用域判别器，用 source/target 域标签计算对抗域损失。
执行反向传播和参数更新。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
构造或使用域判别器，用 source/target 域标签计算对抗域损失。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
保存中间变量，供后续训练、评估或输出步骤使用。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
条件分支，按数据可用性、划分策略或方法变体选择严格路径。
按验证指标选择超参数；无验证集时使用预声明固定参数。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
保存中间变量，供后续训练、评估或输出步骤使用。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标；balanced accuracy 和 macro-F1 用于处理类别不均衡。
构造或使用域判别器，用 source/target 域标签计算对抗域损失。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。
h。
执行反向传播和参数更新。

D:\thesis\code\training\build_dann_conditional_report.py

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
条件分支的备选路径, 仍保持明确规则而不是隐式猜测。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
计算评价指标: balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标: balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标: balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标: balanced accuracy 和 macro-F1 用于处理类别不均衡。
计算评价指标: balanced accuracy 和 macro-F1 用于处理类别不均衡。
构造或使用域判别器, 用 source/target 域标签计算对抗域损失。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
执行反向传播和参数更新。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
空行, 用来分隔逻辑块, 保持代码结构可读。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
保存中间变量, 供后续训练、评估或输出步骤使用。
构造或使用标签分类器 G_y, 用气体标签训练分类目标。
保存中间变量, 供后续训练、评估或输出步骤使用。
返回由函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
mixing metrics, 把可复用的训练、评估或数据处理步骤独立出来。
保存中间变量, 供后续训练、评估或输出步骤使用。
定义时间/域划分, 明确训练、验证、测试样本来源。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量, 供后续训练、评估或输出步骤使用。
条件分支, 按数据可用性、划分策略或方法变体选择严格路径。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
条件分支, 按数据可用性、划分策略或方法变体选择严格路径。
显式或隐式输入输入, 输入或特征不满足预定义限制时直接中止距离约束。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
条件分支, 按数据可用性、划分策略或方法变体选择严格路径。
保存中间变量, 供后续训练、评估或输出步骤使用。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
保存中间变量, 供后续训练、评估或输出步骤使用。
循环遍历 split、模型、超参数、epoch 或样本记录。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。

D:\thesis\code\training\build_dann_conditional_report.py

```

233     rows.append(
234         {
235             "method_group": method_group,
236             "split": key[0],
237             "task": key[1],
238             "variant": key[2],
239             "label": key[3],
240             "centroid_distance_norm": centroid_distance,
241             "neighbor_opposite_domain_rate": neighbor_mixing,
242             "n_source": len(src),
243             "n_target": len(tgt),
244         }
245     )
246     return pd.DataFrame(rows)
247
248
249 def selection_label(split: dict, arrays: dict) -> str:
250     selection = str(split["selection"])
251     if selection == "target_validation_labels":
252         if len(arrays["x"]["target_val"]) == 0:
253             raise ValueError(f"{split['split']} declares target validation selection but target_val is empty.")
254         return "target validation labels"
255     if selection == "source_validation_labels":
256         if len(arrays["x"]["source_val"]) == 0:
257             raise ValueError(f"{split['split']} declares source validation selection but source_val is empty.")
258         return "source validation labels"
259     if selection == dann.NO_VALIDATION_SELECTION:
260         return dann.NO_VALIDATION_SELECTION
261     raise ValueError(f"Unknown split selection policy: {selection}")
262
263
264 def candidate_grid(has_validation: bool) -> list[tuple[float, float, int]]:
265     if not has_validation:
266         return [(DEFAULT_LAMBDA, DEFAULT_ALPHA, DEFAULT_SEED)]
267     return [
268         (float(lambda_value), float(align_strength), int(seed))
269         for lambda_value in dann.LAMBDA_VALUES
270         for align_strength in ALIGN_STRENGTHS
271         for seed in dann.SEEDS
272     ]
273
274
275 def pseudo_dann_row(use_target_labels: bool, lambda_value: float, seed: int, select_label: str) -> pd.Series:
276     return pd.Series(
277         {
278             "lambda": float(lambda_value),
279             "seed": int(seed),
280             "classifier_label_usage": "source labels + selected target labels" if use_target_labels else "source labels only",
281             "model_selection_labels": select_label,
282         }
283     )
284
285
286 def train_improved() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
287     df = pd.read_csv(FEATURE_PATH)
288     summary_rows = []
289     all_curves = []
290     all_features = []

```

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

定义时间/域划分, 明确训练、验证、测试样本来源。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

返回本函数计算出的模型、指标、表格行或中间结果。

空行, 用来分隔逻辑块, 保持代码结构可读。

窄段函数来分隔逻辑块, 保持代码结构可读。

selection_label, 把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分, 明确训练、验证、测试样本来源。

返回本函数计算出的模型、指标、表格行或中间结果。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

定义时间/域划分, 明确训练、验证、测试样本来源。

返回本函数计算出的模型、指标、表格行或中间结果。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

返回本函数计算出的模型、指标、表格行或中间结果。

显式失败保护: 输入、划分或特征不满足实验假设时直接停止。

空行, 用来分隔逻辑块, 保持代码结构可读。

窄段函数来分隔逻辑块, 保持代码结构可读。

candidate_grid, 把可复用的训练、评估或数据处理步骤独立出来。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

返回本函数计算出的模型、指标、表格行或中间结果。

返回本函数计算出的模型、指标、表格行或中间结果。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

循环遍历 split、模型、超参数、epoch 或样本记录。

循环遍历 split、模型、超参数、epoch 或样本记录。

循环遍历 split、模型、超参数、epoch 或样本记录。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

空行, 用来分隔逻辑块, 保持代码结构可读。

窄段函数来分隔逻辑块, 保持代码结构可读。

pseudo_dann_row, 把可复用的训练、评估或数据处理步骤独立出来。

返回本函数计算出的模型、指标、表格行或中间结果。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

构造或使用标签分类器 G_y, 用气体标签训练分类目标。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

空行, 用来分隔逻辑块, 保持代码结构可读。

窄段函数来分隔逻辑块, 保持代码结构可读。

train_improved, 把可复用的训练、评估或数据处理步骤独立出来。

指定输入特征表、原始数据输入或输出表格/图像位置。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

Chapter 4 - Custom C-DANN centroid alignment

D:\thesis\code\training\build_dann_conditional_report.py

```
291
292     variants = [
293         ("DANN-UDA", "C-DANN-UDA", False),
294         ("DANN-semi", "C-DANN-semi", True),
295     ]
296     jobs = []
297     for split in dann.SPLITS:
298         for task_name, classes in dann.REPORT_TASKS:
299             for old_variant, new_variant, use_target_labels in variants:
300                 jobs.append((split, task_name, classes, old_variant, new_variant, use_target_labels))
301
302     for i, (split, task_name, classes, old_variant, new_variant, use_target_labels) in enumerate(jobs):
303         task, parts = base.prepare_parts(df, split, classes)
304         arrays = base.make_arrays(task, parts, classes)
305         if use_target_labels and len(arrays["x"]["target_labeled_train"]) == 0:
306             continue
307         candidates = []
308         select_label = selection_label(split, arrays)
309         has_validation = select_label != dann.NO_VALIDATION_SELECTION
310         print(f"({i + 1:02d}/{len(jobs)}) {split['split']} | {task_name} | {new_variant}")
311         for lambda_value, align_strength, seed in candidate_grid(has_validation):
312             row_for_training = pseudo_dann_row(use_target_labels, lambda_value, seed, select_label)
313             model, curves, info = train_conditional(row_for_training, arrays, classes, align_strength)
314             x_cls, y_cls, classifier_label_usage = base.classifier_data(arrays, row_for_training)
315             summary_row = pd.Series(
316                 {
317                     "split": split["split"],
318                     "description": split["description"],
319                     "task": task_name,
320                     "variant": new_variant,
321                     "lambda": lambda_value,
322                     "seed": seed,
323                     "classifier_train_n": len(y_cls),
324                     "source_train_n": len(arrays["y"]["source_train"]),
325                     "source_val_n": len(arrays["y"]["source_val"]),
326                     "target_labeled_train_n": len(arrays["y"]["target_labeled_train"]),
327                     "target_domain_n": len(arrays["x"]["target_domain"]),
328                     "target_validation_n": len(arrays["y"]["target_val"]),
329                     "target_test_n": len(arrays["y"]["target_test"]),
330                     "domain_label_usage": "source/target domain labels only; gas labels not used for domain loss",
331                     "feature_extractor_objective": "min L - lambda*Ld + alpha*Lc via GRL and class-wise centroid alignment",
332                 }
333             )
334             summary_row["classifier_label_usage"] = info["classifier_label_usage"]
335             summary_row["model_selection_labels"] = info["model_selection_labels"]
336             summary_row["conditional_alignment_strength"] = align_strength
337             summary_row["selection_status"] = "validation-selected best hyperparameters" if has_validation else "no validation; fixed
... default hyperparameters"
338         for metric, value in info.items():
339             if metric not in {"classifier_label_usage", "model_selection_labels"}:
340                 summary_row[metric] = value
341             summary_row["train_test_accuracy_gap"] = info["train_accuracy"] - info["test_accuracy"]
342             summary_row["judgment"] = summary_row["selection_status"]
343             candidate = (summary_row, curves, model, align_strength)
344             candidates.append(candidate)
345
346     def score(candidate: tuple[pd.Series, list[dict], dann.DANN, float]) -> tuple[float, float, float, float, float]:
347         row = candidate[0]
```

空行, 用来分隔逻辑块, 保持代码结构可读。

保存中间变量, 供后续训练、评估或输出步骤使用。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

保存中间变量, 供后续训练、评估或输出步骤使用。

定义时间/域划分, 明确训练、验证、测试样本来源。

循环遍历 split、模型、超参数、epoch 或样本记录。

循环遍历 split、模型、超参数、epoch 或样本记录。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

空行, 用来分隔逻辑块, 保持代码结构可读。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

条件分支, 按数据可用性、划分策略或方法变体选择严格路径。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

定义时间/域划分, 明确训练、验证、测试样本来源。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

构造或使用标签分类器 G_y, 用气体标签训练分类目标。

保存中间变量, 供后续训练、评估或输出步骤使用。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

定义时间/域划分, 明确训练、验证、测试样本来源。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

构造或使用标签分类器 G_y, 用气体标签训练分类目标。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

构造或使用特征提取器 G_f, 将输入特征映射到潜在表示 h。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

构造或使用标签分类器 G_y, 用气体标签训练分类目标。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

上一行过长后的续行, 保持同一条代码的剩余内容。

循环遍历 split、模型、超参数、epoch 或样本记录。

构造或使用标签分类器 G_y, 用气体标签训练分类目标。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

保存中间变量, 供后续训练、评估或输出步骤使用。

C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

空行, 用来分隔逻辑块, 保持代码结构可读。

定义函数 score, 把可复用的训练、评估或数据处理步骤独立出来。

保存中间变量, 供后续训练、评估或输出步骤使用。

D:\thesis\code\training\build_dann_conditional_report.py

```

348         return (
349             float(row["validation_macro_f1"]),
350             float(row["validation_balanced_accuracy"]),
351             float(row["validation_accuracy"]),
352             -float(row["lambda"]),
353             -float(row["seed"]),
354         )
355
356     best_row, best_curves, best_model, best_align = candidates[0] if not has_validation else sorted(candidates, key=score,
357     ... reverse=True)[0]
358     summary_rows.append(best_row.to_dict())
359     for curve in best_curves:
360         curve.update(
361             {
362                 "split": best_row["split"],
363                 "task": best_row["task"],
364                 "variant": best_row["variant"],
365                 "lambda": float(best_row["lambda"]),
366                 "seed": int(best_row["seed"]),
367                 "conditional_alignment_strength": best_align,
368             }
369         )
370     all_curves.extend(best_curves)
371     all_features.extend(
372         base.build_feature_rows(str(best_row["split"]), str(best_row["task"]), str(best_row["variant"]), best_model, parts, arrays)
373     )
374
375     return pd.DataFrame(summary_rows), pd.DataFrame(all_curves), pd.DataFrame(all_features)
376
377 def hide_summary_pseudo_validation(summary: pd.DataFrame) -> pd.DataFrame:
378     summary = summary.copy()
379     no_validation = summary["model_selection_labels"].eq(dann.NO_VALIDATION_SELECTION)
380     for col in ["validation_accuracy", "validation_balanced_accuracy", "validation_macro_f1"]:
381         if col in summary:
382             summary.loc[no_validation, col] = np.nan
383     return summary
384
385
386 def compare_old_new(old_summary: pd.DataFrame, new_summary: pd.DataFrame, old_mix: pd.DataFrame, new_mix: pd.DataFrame) -> pd.DataFrame:
387     pairs = {"C-DANN-UDA": "DANN-UDA", "C-DANN-semi": "DANN-semi"}
388     rows = []
389     old_mix_avg = old_mix.groupby(["split", "task", "variant"]).agg(
390         old_centroid_distance=("centroid_distance_norm", "mean"),
391         old_neighbor_mixing=("neighbor_opposite_domain_rate", "mean"),
392     )
393     new_mix_avg = new_mix.groupby(["split", "task", "variant"]).agg(
394         new_centroid_distance=("centroid_distance_norm", "mean"),
395         new_neighbor_mixing=("neighbor_opposite_domain_rate", "mean"),
396     )
397     for _, new in new_summary.iterrows():
398         old_variant = pairs[str(new["variant"])]
399         old = old_summary[
400             old_summary["split"].eq(new["split"])
401             & old_summary["task"].eq(new["task"])
402             & old_summary["variant"].eq(old_variant)
403         ].iloc[0]
404         old_m = old_mix_avg.loc[(old["split"], old["task"], old["variant"])]

```

返回本函数计算出的模型、指标、表格行或中间结果。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。

计算评价指标：balanced accuracy 和 macro-F1 用于处理类别不均衡。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

上一行过长后的续行，保持同一条代码的剩余内容。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

循环遍历 split、模型、超参数、epoch 或样本记录。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

定义时间/域划分，明确训练、验证、测试样本来源。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

定义时间/域划分，明确训练、验证、测试样本来源。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

空行，用来分隔逻辑块，保持代码结构可读。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

返回函数来分隔逻辑块，保持代码结构可读。把可复用的训练、评估或数据预处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

循环遍历 split、模型、超参数、epoch 或样本记录。

条件分支，按数据可用性、划分策略或方法变量选择严格路径。

保存中间变量，供后续训练、评估或输出步骤使用。

返回本函数计算出的模型、指标、表格行或中间结果。

空行，用来分隔逻辑块，保持代码结构可读。

返回函数来分隔逻辑块，保持代码结构可读。

compare old new，把可复用的训练、评估或数据预处理步骤独立出来。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分条件明确训练、验证、测试样本来源域同类中心距离约束 h。

保存中间变量，供后续训练、评估或输出步骤使用。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

定义时间/域划分条件明确训练、验证、测试样本来源域同类中心距离约束 h。

保存中间变量，供后续训练、评估或输出步骤使用。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

循环遍历 split、模型、超参数、epoch 或样本记录。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

定义时间/域划分，明确训练、验证、测试样本来源。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。

定义时间/域划分，明确训练、验证、测试样本来源。

Chapter 4 - Custom C-DANN centroid alignment

D:\thesis\code\training\build_dann_conditional_report.py

```
405     new_m = new_mix_avg.loc[(new["split"], new["task"], new["variant"])]
406     test_delta = float(new["test_accuracy"]) - float(old["test_accuracy"])
407     centroid_delta = float(new_m["new_centroid_distance"]) - float(old_m["old_centroid_distance"])
408     mixing_delta = float(new_m["new_neighbor_mixing"]) - float(old_m["old_neighbor_mixing"])
409     rows.append(
410         {
411             "split": new["split"],
412             "task": new["task"],
413             "old_variant": old_variant,
414             "new_variant": new["variant"],
415             "old_test_accuracy": old["test_accuracy"],
416             "new_test_accuracy": new["test_accuracy"],
417             "test_accuracy_delta": test_delta,
418             "old_domain_accuracy": old["domain_discriminator_accuracy"],
419             "new_domain_accuracy": new["domain_discriminator_accuracy"],
420             "old_centroid_distance": old_m["old_centroid_distance"],
421             "new_centroid_distance": new_m["new_centroid_distance"],
422             "centroid_distance_delta": centroid_delta,
423             "old_neighbor_mixing": old_m["old_neighbor_mixing"],
424             "new_neighbor_mixing": new_m["new_neighbor_mixing"],
425             "neighbor_mixing_delta": mixing_delta,
426         }
427     )
428     return pd.DataFrame(rows)
429
430
431 def svg_escape(value: object) -> str:
432     return (
433         str(value)
434         .replace("&", "&amp;")
435         .replace("<", "&lt;")
436         .replace(">", "&gt;")
437         .replace("'", "&quot;")
438     )
439
440
441 def fmt_pct(value: object) -> str:
442     if pd.isna(value):
443         return "NA"
444     return f"{100 * float(value):.1f}%"
445
446
447 def fmt_delta(value: object, signed: bool = True) -> str:
448     if pd.isna(value):
449         return "NA"
450     sign = "+" if signed and float(value) > 0 else ""
451     return f"{sign}{float(value):.3f}"
452
453
454 def write_comparison_svg(comparison: pd.DataFrame) -> None:
455     rows = comparison.copy().sort_values(["split", "task", "old_variant"]).reset_index(drop=True)
456     row_h = 29
457     header_h = 170
458     width = 1800
459     height = header_h + 44 + max(len(rows), 1) * row_h + 110
460     col_x = [35, 300, 610, 815, 990, 1165, 1335, 1505, 1665]
461     headers = [
462         "Split",
```

定义时间/域划分, 明确训练、验证、测试样本来源。
保存DANN在训练条件供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
定义时间/域划分, 明确训练、验证、测试样本来源。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
构造或使用域判别器, 用 source/target 域标签计算对抗域损失。
构造或使用域判别器, 用 source/target 域标签计算对抗域损失。
C-DANN 的类条件 centroid alignment, 用源域/目标域同类中心距离约束
C-DANN 的类条件 centroid alignment, 用源域/目标域同类中心距离约束
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 svg_escape, 把可复用的训练、评估或数据处理步骤独立出来。
返回本函数计算出的模型、指标、表格行或中间结果。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 fmt_pct, 把可复用的训练、评估或数据处理步骤独立出来。
条件分支, 按数据可用性、划分策略或方法变体选择严格路径。
返回本函数计算出的模型、指标、表格行或中间结果。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义函数 fmt_delta, 把可复用的训练、评估或数据处理步骤独立出来。
条件分支, 按数据可用性、划分策略或方法变体选择严格路径。
返回本函数计算出的模型、指标、表格行或中间结果。
保存中间变量, 供后续训练、评估或输出步骤使用。
返回本函数计算出的模型、指标、表格行或中间结果。
空行, 用来分隔逻辑块, 保持代码结构可读。
空行, 用来分隔逻辑块, 保持代码结构可读。
定义时间/域划分, 明确训练、验证、测试样本来源。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
保存中间变量, 供后续训练、评估或输出步骤使用。
C-DANN 训练流程中的结构语句, 服务于 centroid 对齐和结果比较。

D:\thesis\code\training\build_dann_conditional_report.py

```

443 "Task",
444 "Variant",
445 "Test old",
446 "Test C-DANN",
447 "Delta",
448 "Domain old/new",
449 "Centroid delta",
450 "Mixing delta",
451 ]
452 parts = [
453     f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width} {height}">',
454     '<rect width="100%" height="100%" fill="#ffffff"/>',
455     '<text x="35" y="44" font-family="Arial, Helvetica, sans-serif" font-size="28" font-weight="700" fill="#111827">9-feature DANN vs',
456     '... C-DANN Summary</text>',
457     '<text x="35" y="78" font-family="Arial, Helvetica, sans-serif" font-size="15" fill="#4b5563">Features: 9 feature families x 6',
458     '... sensors = 54-dimensional input vector per trial. C-DANN adds class-conditional latent centroid alignment.</text>',
459     '<text x="35" y="105" font-family="Arial, Helvetica, sans-serif" font-size="15" fill="#4b5563">Positive mixing delta and negative',
460     '... centroid delta indicate better source-target mixing in the plotted latent space.</text>',
461     '<rect x="25" y="135" width="1750" height="38" rx="6" fill="#f3f4f6" stroke="#d1d5db"/>',
462 ]
463 for x, header in zip(col_x, headers):
464     parts.append(
465         f'<text x="{x}" y="160" font-family="Arial, Helvetica, sans-serif" font-size="13" font-weight="700"',
466         '... fill="#111827">{svg_escape(header)}</text>'
467     )
468
469 y = header_h + 30
470 for idx, row in rows.iterrows():
471     fill = "#ffffff" if idx % 2 == 0 else "#f9fafb"
472     parts.append(f'<rect x="25" y="{y - 19}" width="1750" height="{row_h}" fill="{fill}" stroke="#eef2f7"/>')
473     values = [
474         row["split"],
475         row["task"],
476         f'{row["old_variant"]} -> {row["new_variant"]}',
477         fmt_pct(row["old_test_accuracy"]),
478         fmt_pct(row["new_test_accuracy"]),
479         fmt_delta(row["test_accuracy_delta"]),
480         f'{fmt_pct(row["old_domain_accuracy"])} / {fmt_pct(row["new_domain_accuracy"])}',
481         fmt_delta(row["centroid_distance_delta"]),
482         fmt_delta(row["neighbor_mixing_delta"]),
483     ]
484     for x, value in zip(col_x, values):
485         parts.append(
486             f'<text x="{x}" y="{y}" font-family="Arial, Helvetica, sans-serif" font-size="12" fill="#111827">{svg_escape(value',
487             '... )}</text>'
488         )
489     y += row_h
490
491 parts += [
492     '<text x="35" y="{y}" font-family="Arial, Helvetica, sans-serif" font-size="13" fill="#374151">This table reports raw deltas only;',
493     '... it does not apply threshold-based pass/fail labels.</text>'.format(height - 48),
494     "</svg>",
495 ]
496 SVG_PATH.write_text("\n".join(parts), encoding="utf-8")
497
498 def main() -> None:
499     torch.set_num_threads(1)

```

[illegible]

D:\thesis\code\training\build_dann_conditional_report.py

```

515 FIG_DIR.mkdir(parents=True, exist_ok=True)
516
517 old_summary = pd.read_csv(OLD_SUMMARY_PATH)
518 old_curves = pd.read_csv(OLD_CURVES_PATH)
519 old_features = pd.read_csv(OLD_FEATURE_SPACE_PATH)
520 partitions = pd.read_csv(PARTITION_PATH)
521
522 new_summary, new_curves, new_features = train_improved()
523 new_summary = hide_summary_pseudo_validation(new_summary)
524 old_mix = mixing_metrics(old_features, "old")
525 new_mix = mixing_metrics(new_features, "conditional")
526 comparison = compare_old_new(old_summary, new_summary, old_mix, new_mix)
527
528 new_summary.to_csv(NEW_SUMMARY_PATH, index=False)
529 new_curves.to_csv(NEW_CURVES_PATH, index=False)
530 new_features.to_csv(NEW_FEATURE_SPACE_PATH, index=False)
531 pd.concat([old_mix, new_mix], ignore_index=True).to_csv(MIXING_PATH, index=False)
532 comparison.to_csv(COMPARE_PATH, index=False)
533 write_comparison_svg(comparison)
534
535 combined_summary = pd.concat([old_summary, new_summary], ignore_index=True, sort=False)
536 combined_curves = pd.concat([old_curves, new_curves], ignore_index=True, sort=False)
537 combined_features = pd.concat([old_features, new_features], ignore_index=True, sort=False)
538 combined_summary = combined_summary[combined_summary["task"].eq("three_class_with_air")].reset_index(drop=True)
539 combined_curves = combined_curves[combined_curves["task"].eq("three_class_with_air")].reset_index(drop=True)
540 combined_features = combined_features[combined_features["task"].eq("three_class_with_air")].reset_index(drop=True)
541 partitions = partitions[partitions["task"].eq("three_class_with_air")].reset_index(drop=True)
542 base.HTML_PATH = HTML_PATH
543 base.write_html(combined_summary, partitions, combined_curves, combined_features)
544
545 print("\nComparison")
546 print(
547     comparison[
548         [
549             "split",
550             "task",
551             "old_variant",
552             "new_variant",
553             "old_test_accuracy",
554             "new_test_accuracy",
555             "test_accuracy_delta",
556             "old_centroid_distance",
557             "new_centroid_distance",
558             "centroid_distance_delta",
559             "old_neighbor_mixing",
560             "new_neighbor_mixing",
561             "neighbor_mixing_delta",
562         ]
563     ].to_string(index=False)
564 )
565 print("\nWrote")
566 print(HTML_PATH)
567 print(SVG_PATH)
568 print(COMPARE_PATH)
569 print(MIXING_PATH)
570
571
572 if name == "main ":

```

指定输入特征表、原始数据目录或输出表格/图像位置。
空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

空行，用来分隔逻辑块，保持代码结构可读。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

保存中间变量，供后续训练、评估或输出步骤使用。

写出结果表、图或日志。供论文图表和复查使用。

空行，用来分隔逻辑块，保持代码结构可读。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

定义时间/域划分、明确训练、验证、测试样本来源。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 的类条件 centroid alignment，用源域/目标域同类中心距离约束

C-DANN 的类条件 centroid alignment，用源域/目标域同类中心距离约束

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

保存中间变量，供后续训练、评估或输出步骤使用。

C-DANN 训流程中的结构语句，服务于 centroid 对齐和结果比较。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

写出结果表、图或日志。供论文图表和复查使用。

空行，用来分隔逻辑块，保持代码结构可读。

空行，用来分隔逻辑块，保持代码结构可读。

条件分支，按数据可用性、代价策略或方法变体选择严格路径。

Chapter 4 - Custom C-DANN centroid alignment

D:\thesis\code\training\build_dann_conditional_report.py

573 `main()`

C-DANN 训练流程中的结构语句，服务于 centroid 对齐和结果比较。